

第 11 卷 签到题百科

考试速查：主查签到题常识：常用数学、微积分、线代、Markov、概率统计、机器学习指标、文件媒体大小、进制补码浮点、读程序流程图、编码网络和生活模拟模板。

Contents

第 11 卷 签到题百科	3
考试速查定位	3
签到题百科使用原则	3
SIGN-00 签到题百科总路由	4
1. 题面关键词路由表	5
2. 超过 160 条覆盖清单	5
3. 考场优先级	6
SIGN-AI-02 AI 原理基础百科	6
1. 概念层级	7
2. 学习范式	7
3. 搜索和规划	7
4. 常见模型	8
5. 神经网络最小概念	8
6. AI 题型路由	8
SIGN-AI-03 现代 AI 术语、向量检索与 Transformer 常识	8
1. 现代 AI 术语翻译表	9
2. 训练、验证、测试	10
3. 向量相似度	10
4. Top-K 检索完整代码	11
5. Softmax	11
6. Attention 最小模型	12
7. Transformer 常识	12
8. RAG 题型拆解	13
9. 生成模型参数常识	13
10. 与 Markov 性质的关系	13
SIGN-ARCH-01 计算机体系结构基础	13
1. 五大部件和层次结构	14
2. CPU 执行模型	14
3. 局部性和缓存	14
4. 字节序	15
5. 性能估算	15
SIGN-ARCH-02 数值表示、补码、浮点、字节序与内存	15
1. 进制速查	16
2. 补码范围	17
3. 位运算常识	17
4. 整数溢出	18
5. 浮点数常识	18
6. 字节序	18
7. 内存区常识	19
8. 常见类型大小	19
9. 内存估算	19
10. 对齐和结构体大小	19
11. 字符编码长度	20
12. BCD 和固定小数	20

SIGN-CALC-01 高等数学、微积分与数值方法	20
1. 常见导数表	21
2. 求导规则	21
3. 数值方法短代码	21
4. 常见原函数和积分	22
5. 优化和机器学习常见式	22
SIGN-COMP-01 复杂度、数据规模与工程估算	22
1. 时间预算表	23
2. 内存估算	23
3. 图存储估算	24
4. 常见优化方向	24
SIGN-CS-01 计算机常识、编码、文件和网络	24
1. 单位表	25
2. BMP 和媒体大小	25
3. 编码和进制	25
4. IPv4 / CIDR	26
SIGN-LA-01 线性代数、向量与小矩阵	26
1. 向量公式	27
2. 小矩阵公式	27
3. 机器学习里常见线代	28
SIGN-LOGIC-01 离散逻辑、布尔代数与自动机	28
1. 逻辑公式	29
2. 集合和关系	29
3. 自动机	29
4. 正则表达式常识	30
SIGN-MARKOV-01 马尔可夫性质、Markov 链与状态转移	30
1. Markov 性质到底是什么	31
2. 转移矩阵	31
3. 直接迭代模板	32
4. 矩阵快速幂模板	32
5. 平稳分布	33
6. 吸收状态	33
7. HMM、MDP、强化学习的关系	33
8. 完整可运行模板	33
9. 最小测试样例	35
10. 考场判断清单	35
SIGN-MATH-01 初等数学、单位与几何公式	35
1. 常用公式清单	36
2. 高频小函数	37
3. 签到题模型补充	37
SIGN-MEDIA-02 多媒体、文件格式与压缩估算	37
1. 图片大小	38
2. 颜色	38
3. 音频大小	39
4. 视频大小	39
5. 压缩率	39
SIGN-ML-01 机器学习算法常识与小模拟	39
1. 监督学习和评估指标	40
2. 常见模型速查	40
3. softmax 和 kNN 短代码	41
SIGN-NOIP-01 NOIP/CSP 初赛式信息学常识	41
1. 初赛常识速查表	42
2. 复杂度判断口令	42
3. 读程序常见语义	43
4. 数据结构概念题	43

5. 排序常识	43
SIGN-NOIP-02 读程序、流程图、伪代码与基础程序语义	44
1. 流程图符号	45
2. 变量表模板	45
3. 循环次数速查	45
4. C++ 运算符优先级高频版	45
5. 自增自减	46
6. 短路求值	46
7. 函数传参	46
8. 递归调用栈	47
9. 递归输出顺序	47
10. 伪代码翻译规则	47
11. 分支配对	48
12. 二维数组和循环顺序	48
13. 读程序完整示例	48
14. 看输出格式	48
15. 代码阅读排错清单	49
SIGN-OSNET-01 操作系统、网络与数据库常识	49
1. 操作系统常识	50
2. 网络常识	50
3. 数据库和 SQL	50
SIGN-PROB-01 概率统计速查	51
1. 概率公式	52
2. 描述统计小代码	52
3. 相关系数	52
4. 常见分布	52
SIGN-SEC-01 信息安全、校验、压缩与信息论常识	53
1. 概念区分	53
2. 校验短代码	54
3. 压缩和熵	54
4. 常见压缩率	54
SIGN-SIM-01 生活化签到模拟题模板	54
1. 高频短规则	55
2. 完整可运行小工具	55
3. 最小测试样例	58

第 11 卷 签到题百科

考试速查定位

第 11 卷 签到题百科

- **这是第几卷：** 11。
- **主要查什么：** 主查签到题常识：常用数学、微积分、线代、Markov、概率统计、机器学习指标、文件媒体大小、进制补码浮点、读程序流程图、编码网络和生活模拟模板。
- **什么时候翻：** 题面是公式、单位、图片大小、日期、统计指标、机器学习小模拟或计算机常识时翻。
- **题面关键词：** BMP；三角形面积；单位换算；微积分；线性代数；Markov；概率统计；F1；kNN；bit/byte；补码；浮点；流程图；AI 术语；日期；Excel 列号。

自动由 SIGN 模块重建。定位是常用数学、计算机常识、微积分、线性代数、概率统计、机器学习和生活模拟签到题。

签到题百科使用原则

题面信号	先翻模块
初赛/CSP 概念、读程序、稳定排序、数据结构概念	SIGN-NOIP-01
读程序、流程图、递归栈、伪代码、运算符优先级	SIGN-NOIP-02
CPU、Cache、内存层次、性能估算	SIGN-ARCH-01
进制、补码、浮点、字节序、内存和对齐	SIGN-ARCH-02
复杂度、内存限制、数量级估算	SIGN-COMP-01

题面信号	先翻模块
三角形、面积、折扣、等差等比、单位换算	SIGN-MATH-01
导数、积分、梯度、数值近似	SIGN-CALC-01
向量、矩阵、距离、投影、Markov	SIGN-LA-01
Markov 性质、转移矩阵、平稳分布、HMM/MDP	SIGN-MARKOV-01
命题逻辑、集合关系、自动机、正则常识	SIGN-LOGIC-01
均值、方差、概率、Bayes、相关系数	SIGN-PROB-01
AI 原理、监督/无监督/强化学习、神经网络概念	SIGN-AI-02
Token、Embedding、Attention、Transformer、RAG	SIGN-AI-03
混淆矩阵、F1、kNN、k-means、softmax	SIGN-ML-01
bit/byte、BMP、音频视频、IP、编码	SIGN-CS-01
图片、音频、视频、颜色、压缩率	SIGN-MEDIA-02
操作系统、网络、数据库、Web、SQL	SIGN-OSNET-01
安全、哈希、校验、压缩、信息熵	SIGN-SEC-01
BMI、排名、Excel 列号、日期差、括号	SIGN-SIM-01

这卷的使用方式：先查公式和单位，再写小函数；不要为了签到题临场推导。复杂算法仍回到前面对应卷。

SIGN-00 签到题百科总路由

模块编号：SIGN-00

模块名称：签到题百科总路由：常用数学、计算机常识和生活模拟

标签：签到题、常识题、公式题、模拟、数学、计算机基础、C++17、速查

一句话用途：遇到看起来不像经典算法模板、而是考公式、单位、格式、常识或按规则计算的题时，先用本模块把题面关键词路由到第 11 卷对应页。

题面触发词：

- 图片大小、BMP、RGB、DPI、音频采样、码率、下载时间。
- 三角形面积、角度弧度、速度时间路程、折扣、税率、复利。
- 导数、积分、梯度、矩阵、概率、方差、混淆矩阵、F1。
- bit、byte、ASCII、UTF-8、补码、IP、CIDR、Base64、URL 编码。
- BMI、GPA、排名、Excel 列号、日期差、星期、时区。

什么时候用：

- 题目数据范围很小，核心是套公式或按规则模拟。
- 题面含现实对象、文件格式、统计指标、机器学习指标、单位换算。
- 你感觉“应该很简单”，但公式、边界或单位可能记不准。

不要什么时候用：

- 已经明显是图论、DP、数据结构、字符串匹配等算法题，应回到对应卷。
- 题目要求完整 JSON/表达式/解释器，优先 SIM-03/04/05。
- 方程求解、数值求根和多项式计算，优先 SIM-07。

复杂度：

- 本卷多数公式题是 $O(1)$ 。
- 统计、转换、扫描类通常是 $O(n)$ 。
- 小矩阵、小数据机器学习模拟通常是 $O(n*d)$ 或 $O(n*d*k)$ 。

依赖的标准容器：

- string：格式、编码、进制、列号。
- vector<double> / 静态数组：统计、向量、矩阵、小规模机器学习。
- map / unordered_map：频次统计、类别映射。
- queue / stack：括号、自动机、简单状态模拟。

输入如何整理：

1. 先判断是公式题、格式题、统计题还是现实模拟题。
2. 圈单位：bit/byte、秒/分钟、度/弧度、KB/KiB、bps/B/s。
3. 圈输出：整数、保留小数、百分比、四舍五入、向下取整。
4. 圈边界：0、负数、闰年、前导零、空字符串、同分排名。

接口：

- SIGN-00 先路由。
- SIGN-MATH-01 查初等数学和单位。
- SIGN-CALC-01 查微积分和数值方法。
- SIGN-LA-01 查向量、矩阵、线性代数。
- SIGN-PROB-01 查概率统计。
- SIGN-ML-01 查机器学习指标和小模型模拟。
- SIGN-CS-01 查计算机常识、编码、文件和网络。
- SIGN-SIM-01 查生活化模拟题短模板。

常见坑：

- KB 可能是 1000 byte, KiB 才是 1024 byte, 题面不明时按题面样例。
- 网络带宽常用 bps, 文件大小常用 B, 下载时间要除以 8。
- BMP 每行通常 4 字节对齐, 24 位图每像素 3 字节, 不是简单 $w \times h \times 3$ 。
- 角度三角函数要先转弧度。
- 保留小数是输出格式, 不等于内部先四舍五入。
- 百分点和百分比不同: 从 20% 到 30% 是增加 10 个百分点, 也是相对增长 50%。

暴力/部分替代：

- 公式忘了但数据小: 直接模拟或枚举。
- 日期公式忘了: 从较早日期一天一天加, 拿小范围分。
- 统计指标忘了: 先输出准确率、计数矩阵等基础量。
- 几何公式忘了: 坐标题可用叉积, 三边题可枚举或套海伦公式。

1. 题面关键词路由表

题面信号	先翻模块	关键提醒
三角形三边、面积、合法性	SIGN-MATH-01	先判 $a+b>c$, 海伦公式用 double
速度、路程、工作效率、折扣、税率	SIGN-MATH-01	单位统一后再算
向上取整、四舍五入、保留小数	SIGN-MATH-01	整数向上取整用 $(a+b-1)/b$
导数、梯度、牛顿法、积分近似	SIGN-CALC-01	迭代要设次数和 EPS
矩阵乘、点积、叉积、投影	SIGN-LA-01	坐标题优先用向量公式
均值、中位数、方差、分位数	SIGN-PROB-01	样本方差分母是 $n-1$
Bayes、条件概率、期望、Markov	SIGN-PROB-01	概率相乘前确认独立
accuracy、precision、recall、F1	SIGN-ML-01	分母为 0 时按题意, 常见输出 0
kNN、k-means、softmax、交叉熵	SIGN-ML-01	数据小, 按公式模拟
bit/byte、补码、ASCII、UTF-8	SIGN-CS-01	注意 signed/unsigned
BMP、音频、视频、带宽、IP	SIGN-CS-01	注意行对齐、bps 和 B/s
BMI、GPA、排名、Excel 列号	SIGN-SIM-01	多数是分支和字符串转换
日期差、星期、时区	SIGN-SIM-01 / SIM-06	简单题翻 SIGN, 复杂历法翻 SIM-06

2. 超过 160 条覆盖清单

编号段	内容	必须配代码的高频点
001-010	常数、角度、log、取整、百分比、像素、科学计数法	角度弧度、取整、公式代入
011-020	等差等比、阶乘、极限、泰勒、级数、收敛	迭代到收敛
021-030	导数、偏导、梯度、Hessian、凸性、牛顿法、梯度下降	牛顿法、有限差分
031-040	积分、梯形、Simpson、Euler 法、函数平	数值积分、ODE Euler
041-050	均值	
041-050	向量、点积、叉积、范数、投影、旋转、齐次坐标	距离、叉积、旋转
051-060	矩阵乘、行列式、逆、秩、特征值、Markov、最小二乘	小矩阵乘、2x2 逆
061-070	条件概率、Bayes、独立、期望、方差、二项、Poisson	Bayes、二项概率
071-080	z-score、Monte Carlo、Markov 链、分位数、协方差、相关系数	相关系数、分位数

编号段	内容	必须配代码的高频点
081-090	均值、中位数、众数、方差、IQR、直方图、A/B 测试	描述统计
091-100	特征、归一化、标准化、one-hot、混淆矩阵、F1、AUC	指标计算
101-110	kNN、距离、朴素贝叶斯、回归、sigmoid、SVM、Gini、k-means	kNN、k-means
111-120	ReLU、softmax、交叉熵、全连接、反传、Q-learning	stable softmax
121-130	bit、byte、进制、补码、浮点、ASCII、UTF-8、位掩码、checksum	进制、ASCII、校验
131-140	文件路径、MIME、CSV、线程、栈堆、缓存、IPv4、CIDR、HTTP、SQL	IPv4/CIDR
141-150	闰年、日期差、时区、Base64、URL 编码、HTML entity、RGB/HSV、图片大小	BMP/媒体大小
151-160	BMI、单位换算、折扣、GPA、排名、Excel 列号、括号、自动机	生活模拟

3. 考场优先级

1. 先翻 SIGN-00 路由。
2. 如果题面是“真实世界对象”，先查单位和格式。
3. 如果有公式，先写小函数，不要把公式散落在 solve() 里。
4. 如果输出保留小数，统一 fixed << setprecision(k)。
5. 如果答案可能溢出，先用 long long，乘法用 __int128 或 long double。

SIGN-AI-02 AI 原理基础百科

模块编号: SIGN-AI-02

模块名称: AI 原理基础: 搜索、监督学习、无监督学习、神经网络、强化学习和评估

标签: 签到题、AI 原理、人工智能、机器学习、神经网络、强化学习、搜索、C++17

一句话用途: AI 专项招生可能把 AI 基础概念出成判断、填空或简单公式模拟，本模块提供从概念到考场实现的最小知识图谱。

题面触发词: 人工智能、机器学习、深度学习、监督学习、无监督学习、强化学习、搜索、启发式、模型、训练、推理、评估。

什么时候用:

- 题目问 AI 概念之间的关系。
- 题目给小模型公式，要求算一次预测或更新。
- 题目是 AI 背景但实际是搜索、统计、排序、DP 或模拟。

不要什么时候用:

- 需要完整 AI 模板代码时，翻第 10 卷。
- 需要机器学习指标短代码时，翻 SIGN-ML-01。
- 不要使用任何第三方库或联网模型。

复杂度:

- 概念判断 $O(1)$ 。
- 小模型前向 $O(\text{层数} * \text{矩阵大小})$ 。
- 搜索类按状态数和边数。

依赖的标准容器: vector<double>、queue、priority_queue、map。

输入如何整理:

AI 题先拆壳:

搜索/规划 -> 图搜索。

分类/回归 -> 公式和统计。

聚类 -> 距离和迭代。

神经网络 -> 矩阵乘和激活函数。

强化学习 -> 状态、动作、奖励、转移。

接口：

AI 关键词 -> 算法本质 -> 对应模块。

概念题 -> 查关系表。

公式题 -> 按题面逐步模拟。

常见坑：

- AI 题不等于要写神经网络，大量题只是搜索、图论、统计。
- 训练是更新参数，推理是用已有参数预测。
- 监督学习有标签，无监督学习无标签，强化学习有奖励。
- 深度学习是机器学习的一类，机器学习是 AI 的一类。

暴力/部分替代：

- 分类不会写复杂模型时，用最近邻或多数类。
- 强化学习不会最优策略时，按给定策略模拟。
- 搜索不会 A^* 时，用 BFS/Dijkstra 拿部分分。

1. 概念层级

人工智能 AI

-> 搜索与规划

-> 机器学习 ML

-> 监督学习

-> 无监督学习

-> 强化学习

-> 深度学习 DL

概念	一句话
AI	让机器表现出智能行为
ML	从数据中学习规律
DL	多层神经网络为主的 ML
训练	用数据更新参数
推理	用模型给新输入输出结果
泛化	对没见过数据表现好
过拟合	训练集好、测试集差
欠拟合	模型太弱，训练也差

2. 学习范式

类型	输入	输出/目标	例子
监督学习	x, y	学 $x \rightarrow y$	分类、回归
无监督学习	只有 x	找结构	聚类、降维
强化学习	状态、动作、奖励	学策略	游戏、路径决策
半监督	少量标签 + 大量无标签	利用无标签数据	文本分类
自监督	从数据自己构造标签	表征学习	预测被遮住词

3. 搜索和规划

名称	本质
BFS	无权最短路
Dijkstra	非负权最短路
A^*	$f=g+h$ 的启发式搜索
Minimax	对手也最优的博弈搜索
Alpha-Beta	Minimax 剪枝
MDP	马尔可夫决策过程

A^* 提醒：

- g 是已走代价。
- h 是估计剩余代价。
- 若 h 不超过真实剩余代价， A^* 更容易保证最优。

4. 常见模型

模型	考场公式
kNN	距离最近的 k 个投票
朴素贝叶斯	$\text{argmax}_y P(y) \cdot \prod P(x_i y)$
线性回归	$y = w \cdot x + b$
Logistic	$p = \text{sigmoid}(w \cdot x + b)$
SVM	最大间隔, hinge loss
决策树	按特征划分, 降低不纯度
k-means	最近中心分配, 重算中心
神经网络	层层矩阵乘 + 激活
HMM	隐状态 + 观测, Viterbi
Q-learning	基于奖励更新动作价值

5. 神经网络最小概念

概念	说明
neuron	加权求和再激活
weight	权重参数
bias	偏置
activation	ReLU/sigmoid/tanh
loss	预测和真实的差距
gradient	参数改变对 loss 的影响
backprop	链式法则从后往前传梯度
epoch	全数据训练一轮
batch	一小批样本
learning rate	每次更新步长

6. AI 题型路由

题面	实际模块
机器人走路、智能体规划	图 BFS/Dijkstra/A*
棋类决策	搜索、博弈 DP
文档相似	字符串、词频、向量
分类指标	SIGN-ML-01
小神经网络前向	第 10 卷 AI 前向传播
反向传播/自动求导	第 10 卷 AI-14/15
SPJ 得分函数	baseline + 指标模拟

SIGN-AI-03 现代 AI 术语、向量检索与 Transformer 常识

模块编号: SIGN-AI-03

模块名称: 现代 AI 术语速查: Token、Embedding、Attention、Transformer、RAG、向量相似度和训练评估

标签: 签到题、AI 原理、现代 AI、Token、Embedding、Attention、Transformer、RAG、向量检索、C++17

一句话用途: 题面出现大模型、向量检索、注意力机制、RAG、嵌入向量、训练/推理等现代 AI 词汇时, 用本模块把概念翻译成可计算的字符串、向量、矩阵、排序和统计问题。

题面触发词:

- token、词元、分词、词表、embedding、词向量、向量相似度。
- attention、self-attention、query/key/value、Transformer。
- RAG、检索增强生成、向量数据库、Top-K 相似文档。
- 训练集、验证集、测试集、数据泄漏、泛化、过拟合。
- 参数、超参数、batch、epoch、learning rate、optimizer。
- 大语言模型、prompt、上下文窗口、幻觉、temperature。

什么时候用：

- 题目要求判断 AI 术语关系，或者给小向量/小矩阵要求算一次结果。
- 题目给多个文本或向量，要求按相似度排序或选 Top-K。
- 题目描述 RAG/推荐系统/检索系统，但实质是字符串处理、词频统计、余弦相似度和排序。
- 题目要求模拟 attention 的一小步计算。

不要什么时候用：

- 如果题面只是普通分类指标，优先翻 SIGN-ML-01。
- 如果题面给完整神经网络和梯度，优先翻第 10 卷的前向传播、反向传播和自动求导。
- 不要现场实现真实深度学习训练框架；上机题通常只会考小规模公式模拟。
- 不要使用第三方库、联网模型或在线 API。

复杂度：

- 文本分词和计数： $O(\text{总字符数})$ 或 $O(\text{总词数})$ 。
- 向量余弦相似度：每对 $O(d)$ 。
- Top-K 检索：排序 $O(n \log n)$ ，堆 $O(n \log k)$ 。
- 简化 attention： $O(n^2 \cdot d)$ ，其中 n 是 token 数， d 是向量维度。

依赖的标准容器：

- string：文本和 token。
- map<string,int> 或 unordered_map<string,int>：词频、词表编号。
- vector<double>：向量。
- priority_queue：Top-K。
- 静态数组：小矩阵计算时更容易 1-index。

输入如何整理：

1. 文本题：先决定分隔方式，是按空格、字符、还是题面给定规则分词。
2. 向量题：把每个样本编号为 $1..n$ ，每个维度编号为 $1..d$ 。
3. 检索题：对每个候选算 score，再排序或用堆取 Top-K。
4. Transformer/attention 题：按题面公式逐步算 Q、K、V、score、softmax、加权和。

接口：

cosine(a,b,d) -> 两个向量余弦相似度。

softmax(x,n) -> 把分数转成概率。

top_k_by_score(items,k) -> 取分数最大的 k 项。

attention_one_query(q,K,V,n,d) -> 单个 query 对 n 个 key/value 做注意力。

常见坑：

- embedding 不是一个词本身，而是词/文本对应的数值向量。
- 余弦相似度要除以两个向量长度；零向量要特判。
- softmax 要减去最大值防止 exp 溢出。
- 训练集用于学习参数，验证集用于调超参数，测试集用于最终评估。
- 数据泄漏是把测试信息提前用于训练或调参，会导致虚高分。
- temperature 越大输出越随机，越小越确定；但竞赛题一般只要求按题面公式模拟。
- RAG 本身不神秘：先检索相关资料，再把资料放入生成模型上下文。

暴力/部分替代：

- 不会复杂 embedding 时，用词频向量、0/1 出现向量或 Jaccard 相似度。
- Top-K 不会堆时，直接排序。
- attention 不会矩阵化时，按三层循环逐项算。
- 题面数据很小，直接用 map<string,int> 和 vector<double>，常数足够。

1. 现代 AI 术语翻译表

术语	考场理解
token	模型处理的最小单位，可能是字、词、子词或符号
tokenizer	把文本切成 token 的规则或算法
vocabulary	token 到编号的映射表
embedding	token/文本/图片映射成向量
context window	一次能放进模型的 token 上限
prompt	输入给模型的指令和上下文
inference	用训练好的模型预测

术语	考场理解
pretraining	大规模预训练
fine-tuning	在特定数据上继续训练
alignment	让模型行为更符合人类偏好
hallucination	生成看似合理但不真实的内容
RAG	检索资料 + 生成回答
vector database	存向量并做相似度搜索
multimodal	文本、图像、音频等多模态

2. 训练、验证、测试

集合	用途	能否调参
训练集	更新模型参数	可以
验证集	选模型、调超参数、早停	可以间接用
测试集	最终报告性能	不应参与调参

常见问法:

模型在训练集 99%，测试集 60% -> 过拟合。

训练集和测试集都很低 -> 欠拟合或特征不足。

用测试集挑模型 -> 数据泄漏。

参数和超参数:

名称	例子	谁决定
参数	权重 w、偏置 b	训练学出来
超参数	学习率、层数、kNN 的 k、正则强度	人或搜索过程指定

3. 向量相似度

常用公式:

```
dot(a,b) = sum a[i]*b[i]
cos(a,b) = dot(a,b) / (sqrt(sum a[i]^2) * sqrt(sum b[i]^2))
euclidean(a,b) = sqrt(sum (a[i]-b[i])^2)
manhattan(a,b) = sum abs(a[i]-b[i])
```

什么时候用:

题面	常见距离
文本向量相似	cosine
几何坐标最近	Euclidean
网格/街区距离	Manhattan
集合相似	Jaccard

余弦相似度模板:

```
double cosine(double a[], double b[], int d) {
    double dot = 0, na = 0, nb = 0;
    for (int i = 1; i <= d; i++) {
        dot += a[i] * b[i];
        na += a[i] * a[i];
        nb += b[i] * b[i];
    }
    if (na == 0 || nb == 0) return 0;
    return dot / sqrt(na) / sqrt(nb);
}
```

4. Top-K 检索完整代码

输入 n d k , 再输入查询向量和 n 个候选向量, 输出余弦相似度最高的 k 个编号。

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1005;
const int MAXD = 105;

int n, d, k;
double qv[MAXD], vec[MAXN][MAXD];

double cosine_one(int id) {
    double dot = 0, nq = 0, nv = 0;
    for (int j = 1; j <= d; j++) {
        dot += qv[j] * vec[id][j];
        nq += qv[j] * qv[j];
        nv += vec[id][j] * vec[id][j];
    }
    if (nq == 0 || nv == 0) return 0;
    return dot / sqrt(nq) / sqrt(nv);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> d >> k;
    for (int j = 1; j <= d; j++) cin >> qv[j];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= d; j++) cin >> vec[i][j];
    }

    vector<pair<double,int>> score;
    for (int i = 1; i <= n; i++) score.push_back({cosine_one(i), i});
    sort(score.begin(), score.end(), [](const pair<double,int> &a, const pair<double,int> &b) {
        if (fabs(a.first - b.first) > 1e-12) return a.first > b.first;
        return a.second < b.second;
    });

    for (int i = 0; i < k && i < (int)score.size(); i++) {
        cout << score[i].second << (i + 1 == k || i + 1 == (int)score.size() ? '\n' : ' ');
    }
    return 0;
}
```

最小测试:

输入:

```
3 2 2
1 0
1 0
0 1
1 1
```

输出:

```
1 3
```

5. Softmax

用途:

- 把一组分数转成概率。
- 多分类最后一层常用。
- attention 中把相似度分数转成权重。

稳定写法:

```
void softmax(double x[], double p[], int n) {
    double mx = x[1];
    for (int i = 2; i <= n; i++) mx = max(mx, x[i]);
    double sum = 0;
    for (int i = 1; i <= n; i++) {
        p[i] = exp(x[i] - mx);
        sum += p[i];
    }
    for (int i = 1; i <= n; i++) p[i] /= sum;
}
```

为什么减最大值:

exp(1000) 会溢出。
softmax(x) 与 softmax(x - max(x)) 结果相同。

6. Attention 最小模型

给一个 query q, 多个 key K[i] 和 value V[i]:

```
score[i] = dot(q, K[i]) / sqrt(d)
weight = softmax(score)
out = sum_i weight[i] * V[i]
```

口令:

Q 问 “我要找什么”, K 表示 “我有什么标签”, V 表示 “真正取走的信息”。

简化模板:

```
const int MAXN = 105;
const int MAXD = 105;
double q[MAXD], key[MAXN][MAXD], val[MAXN][MAXD];
double score[MAXN], w[MAXN], out[MAXD];

void attention_one_query(int n, int d) {
    for (int i = 1; i <= n; i++) {
        score[i] = 0;
        for (int j = 1; j <= d; j++) score[i] += q[j] * key[i][j];
        score[i] /= sqrt((double)d);
    }
    softmax(score, w, n);
    for (int j = 1; j <= d; j++) out[j] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= d; j++) out[j] += w[i] * val[i][j];
    }
}
```

7. Transformer 常识

部件	作用
token embedding	把 token 编号变成向量
positional encoding	给序列位置信息
self-attention	每个 token 看其他 token
multi-head	多组 attention 并行关注不同关系
feed-forward	每个位置独立的小网络
residual	残差连接, 缓解训练困难
layer norm	层归一化, 稳定训练

考场判断:

Transformer 的核心不是 RNN, 而是 self-attention。
自注意力可以并行处理序列, 但标准注意力有 $O(n^2)$ 序列长度开销。

8. RAG 题型拆解

RAG 流程:

query -> 编码成向量 -> 检索 Top-K 文档 -> 拼接上下文 -> 生成答案

可能考法:

考法	实际操作
给查询和文档向量, 选最相关文档	余弦相似度排序
给词频, 算相似度	TF/TF-IDF + cosine
给多个候选答案和证据分	按加权得分排序
判断 RAG 优点	可引入外部资料、缓解知识过期
判断 RAG 风险	检索错、上下文太长、仍可能幻觉

9. 生成模型参数常识

参数	含义
temperature	随机性, 越高越发散
top-k	只在概率最高的 k 个 token 中采样
top-p	只在累计概率达到 p 的候选中采样
max tokens	最大生成长度

竞赛题如果给出这些参数, 通常会把概率表也给出, 让你按规则采样或选最大概率。若题目要求确定输出, 一般会规定取概率最大或给随机数序列。

10. 与 Markov 性质的关系

现代 AI 中经常出现“只看当前状态”的简化:

场景	Markov 化方式
强化学习	状态必须包含决策所需信息
文本生成	下一个 token 由当前上下文决定
HMM	隐状态按 Markov 链转移
递归/DP	状态包含未来所需的全部历史摘要

如果题目说“下一步还依赖上一步动作/前两个字符/上一轮输出”, 就把这些历史并入状态, 和 SIGN-MARKOV-01 的升维思想一致。

SIGN-ARCH-01 计算机体系结构基础

模块编号: SIGN-ARCH-01

模块名称: 计算机体系结构: CPU、内存层次、缓存、指令、性能和字节序

标签: 签到题、计算机体系结构、CPU、内存、缓存、指令、字节序、性能估算、C++17

一句话用途: 当题目考 CPU、内存、缓存、字节序、性能单位或程序运行成本时, 用本模块建立正确直觉。

题面触发词: CPU、主频、寄存器、Cache、内存、磁盘、流水线、指令、字节序、局部性、寻址、吞吐量。

什么时候用:

- 题目要求判断计算机组成、内存层级或性能瓶颈。
- 题目问数组行优先、缓存命中、访问模式。
- 题目涉及大小端、二进制表示、对齐、栈堆。

不要什么时候用:

- 题目要求操作系统调度细节, 翻 SIGN-OSNET-01。
- 题目要求 C++ 具体语法, 翻第 1 卷。
- 题目要求硬件电路深入设计, 本卷只提供信息学考试层面常识。

复杂度:

- 概念题 $O(1)$ 。
- 缓存/访问次数估算按循环次数和数据大小计算。

依赖的标准容器：无固定依赖；代码题通常用数组和 `vector`。

输入如何整理：

先统一单位：

$\text{Hz} = \text{cycles/second}$ 。

$1 \text{ ns} = 1e-9 \text{ s}$ 。

1 GB/s 是字节带宽还是 `bit` 带宽要看题面。

接口：

性能估算: $\text{time} = \text{operations} / \text{operations_per_second}$ 。

传输估算: $\text{time} = \text{bytes} / \text{bytes_per_second}$ 。

缓存估算: 按 `cache line` 大小和访问连续性判断。

常见坑：

- 主频高不一定程序快，内存访问和算法复杂度也重要。
- 顺序访问数组通常比随机访问快。
- 二维数组 `C/C++` 按行优先存储，`a[i][j]` 中 `j` 连续变化更友好。
- 大端/小端影响多字节整数在内存里的字节顺序，不影响数值本身。

暴力/部分替代：

- 复杂性能题不会精算时，先按数量级比较。
- 缓存题不会算行数时，先判断顺序访问优于跳跃访问。

1. 五大部件和层次结构

层次	速度	容量	特点
寄存器	最快	最小	CPU 内部
L1/L2/L3 Cache	很快	小	利用局部性
RAM	中等	较大	断电丢失
SSD/HDD	慢	很大	持久存储
网络/外设	更慢	远端	延迟大

冯诺依曼结构：

输入设备 -> 存储器 -> 运算器/控制器 -> 输出设备

程序和数据都存在存储器中。

2. CPU 执行模型

名称	含义
取指	从内存取下一条指令
译码	判断指令类型和操作数
执行	ALU 运算、访存或跳转
寄存器	CPU 内部高速小存储
ALU	算术逻辑单元
PC	程序计数器，指向下一条指令
指令集	CPU 能理解的机器指令集合
流水线	多条指令不同阶段重叠执行
分支预测	猜测 <code>if/跳转</code> 方向减少停顿

3. 局部性和缓存

局部性	例子
时间局部性	一个变量刚访问过，很快又访问
空间局部性	访问 <code>a[i]</code> 后很可能访问 <code>a[i+1]</code>

二维数组访问：

```
int a[1000][1000];

// 更符合 C++ 行优先, 通常更快。
for (int i = 0; i < 1000; i++) {
    for (int j = 0; j < 1000; j++) {
        a[i][j]++;
    }
}

// 跳列访问, 缓存不友好。
for (int j = 0; j < 1000; j++) {
    for (int i = 0; i < 1000; i++) {
        a[i][j]++;
    }
}
```

4. 字节序

以 0x12345678 为例:

字节序	低地址到高地址
大端	12 34 56 78
小端	78 56 34 12

考试提醒:

- 网络字节序通常是大端。
- x86 常见小端。
- 普通 C++ 数值运算不需要关心字节序, 只有按字节读写二进制时才关心。

5. 性能估算

单位	含义
Hz	每秒周期数
GHz	1e9 Hz
FLOPS	每秒浮点运算次数
IOPS	每秒 I/O 操作次数
latency	单次延迟
throughput	吞吐量

估算口令:

如果 1 秒约 1e8 简单操作:

$O(n^2)$, $n=1e5 \rightarrow 1e10$, 危险。

$O(n \log n)$, $n=1e6 \rightarrow$ 约 $2e7$, 通常可试。

SIGN-ARCH-02 数值表示、补码、浮点、字节序与内存

模块编号: SIGN-ARCH-02

模块名称: 计算机数值与内存表示: 进制、补码、移位、浮点、字节序、内存区和对齐

标签: 签到题、计算机组成、数值表示、补码、浮点数、字节序、内存、C++17

一句话用途: 当题目要求二进制/十六进制转换、补码范围、移位、浮点误差、大小端、内存大小或 C++ 对象存储常识时, 用本模块快速判断。

题面触发词:

- 二进制、八进制、十六进制、原码、反码、补码。
- 有符号整数范围、溢出、左移、右移、位运算。
- 浮点数、精度误差、有效数字、IEEE 754。

- 大端、小端、字节序、内存地址。
- 栈、堆、全局区、静态区、Cache、对齐。

什么时候用：

- 题目像计算机组成/体系结构常识题。
- 模拟题中出现二进制协议、文件头、字节解析。
- 签到题要求估算数组占用、图片/音频/内存大小。
- 代码阅读题涉及溢出、移位、整数除法、浮点比较。

不要什么时候用：

- 需要严格解析真实 IEEE 754 位字段时，按题面给出的格式处理。
- 不要依赖 C++ 有符号整数溢出的结果；这是未定义行为。
- 不要用指针强转读浮点二进制，可能违反别名规则；用 `memcpy`。
- 需要操作系统 API 时，不要按本模块硬猜，按题面规则。

复杂度：

- 进制转换：O(位数)。
- 位运算判断：O(1)。
- 内存估算：O(1) 乘法。
- 模拟二进制协议：O(字节数)。

依赖的标准容器：

- `string`：进制表示。
- `unsigned int`、`unsigned long long`：位运算更安全。
- `memcpy`：安全查看浮点/整数字节。
- `io manip`：十六进制输出。

输入如何整理：

1. 先确认单位：bit 还是 byte，KB 是 1024 还是题面指定 1000。
2. 进制题先把每 4 个二进制位映射成 1 个十六进制位。
3. 补码题先确认位数，例如 8 bit、16 bit、32 bit。
4. 内存题统一换算成 byte，再除以 1024 得 KiB/MiB。

接口：

`range_signed(bits)` -> 有符号补码范围。
`range_unsigned(bits)` -> 无符号范围。
`hex_to_binary / binary_to_hex` -> 进制互转。
`safe_float_bits(x)` -> 用 `memcpy` 查看浮点底层字节。
`memory_bytes(count, sizeof_type)` -> 数组占用估算。

常见坑：

- 1 KB 在计算机存储题通常是 1024 B，网络速率题常用十进制，要看题面。
- 1 byte = 8 bit。
- 8 bit 有符号补码范围是 $[-128, 127]$ ，不是 $[-127, 127]$ 。
- `char` 是否有符号与实现有关，竞赛中不要依赖。
- 有符号整数溢出是未定义行为，无符号整数按模 2^{bits} 回绕。
- 浮点数不能用 `==` 判断一般小数结果。
- 小端机器低地址存低字节，大端机器低地址存高字节。

暴力/部分替代：

- 不确定复杂位运算时，把小位数列出真值表。
- 进制转换怕错时，先转十进制再转目标进制。
- 浮点输出不确定时，按题面精度用 `fixed << setprecision(k)`。
- 二进制协议题先写逐字节解析，别急着用结构体强转。

1. 进制速查

进制	前缀习惯	基数
二进制	<code>0b</code>	2
八进制	<code>0</code>	8
十进制	无	10
十六进制	<code>0x</code>	16

四位二进制到十六进制：

二进制	十六进制	二进制	十六进制
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

口令：

二进制转十六进制：从右往左每 4 位一组，不足左补 0。

十六进制转二进制：每个十六进制位展开成 4 位。

2. 补码范围

b 位整数：

类型	范围
无符号	$0 \dots 2^b - 1$
有符号补码	$-2^{(b-1)} \dots 2^{(b-1)}-1$

常见范围：

位数	无符号	有符号补码
8	$0 \dots 255$	$-128 \dots 127$
16	$0 \dots 65535$	$-32768 \dots 32767$
32	$0 \dots 4294967295$	$-2147483648 \dots 2147483647$
64	$0 \dots 2^{64}-1$	$-2^{63} \dots 2^{63}-1$

负数补码口令：

求 -x 的 b 位补码：x 的二进制按位取反，再加 1。

例子：8 bit 中 -5：

5 = 00000101
取反 = 11111010
加 1 = 11111011

3. 位运算常识

表达式	用途
$x \& 1$	判断奇偶
$x \gg 1$	无符号时相当于除以 2 向下取整
$x \ll k$	无符号时乘 2^k ，注意溢出
$x \& (x-1)$	去掉最低位的 1
$x \& -x$	取最低位的 1 对应权值，树状数组常用
$x \mid (1 \ll i)$	把第 i 位设 1
$x \& \sim(1 \ll i)$	把第 i 位清 0
$x \wedge (1 \ll i)$	翻转第 i 位

建议：

```
unsigned int mask = 1u << i;  
unsigned long long big_mask = 1ull << i;
```

不要写：

```
int mask = 1 << 31; // 可能触发符号位问题
```

4. 整数溢出

竞赛结论:

类型	溢出规则
unsigned int	按模 2^{32} 回绕
unsigned long long	按模 2^{64} 回绕
int/long long	有符号溢出是未定义行为

安全乘法判断:

```
long long a, b;
__int128 t = (__int128)a * b;
```

如果只需要取模:

```
long long mul_mod(long long a, long long b, long long mod) {
    return (long long)((__int128)a * b % mod);
}
```

5. 浮点数常识

类型	常见大小	有效十进制位
float	4 byte	约 6-7 位
double	8 byte	约 15-16 位
long double	依环境	通常更高

口令:

0.1、0.2 这类十进制小数通常不能被二进制浮点精确表示。
比较浮点用 $\text{abs}(a-b) < \text{EPS}$ 。

```
const double EPS = 1e-9;
bool equal_double(double a, double b) {
    return fabs(a - b) < EPS;
}
```

输出:

```
cout << fixed << setprecision(6) << ans << "\n";
```

6. 字节序

假设整数 0x12345678 占 4 字节:

字节序	低地址到高地址
大端 big-endian	12 34 56 78
小端 little-endian	78 56 34 12

安全查看本机字节序:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    unsigned int x = 0x12345678u;
    unsigned char b[4];
    memcpy(b, &x, 4);
    for (int i = 0; i < 4; i++) {
        cout << hex << setw(2) << setfill('0') << (int)b[i] << (i == 3 ? '\n' : ' ');
    }
    return 0;
}
```

注意: 这段代码只是查看本机结果。正式题目若指定大端/小端, 要按题面解析, 不要依赖本机。

7. 内存区常识

区域	常见内容
栈 stack	局部变量、函数调用信息
堆 heap	new、动态分配
全局/静态区	全局变量、static 变量
常量区	字符串字面量等

竞赛建议：

大数组尽量开全局静态数组，避免栈爆。

递归太深可能栈爆，图 DFS 深度可能到 $2e5$ 时考虑迭代写法。

例子：

```
const int MAXN = 200000 + 5;
int a[MAXN]; // 全局数组，通常比函数内大数组更稳
```

8. 常见类型大小

实际大小由环境决定，但 ACM/C++17 常见为：

类型	常见大小
char	1 byte
short	2 byte
int	4 byte
long long	8 byte
float	4 byte
double	8 byte
指针	64 位环境 8 byte

考场安全做法：

```
cout << sizeof(int) << "\n";
```

但实际竞赛题通常按标准常见大小给条件，不要求现场探测。

9. 内存估算

公式：

数组字节数 = 元素个数 * 每个元素字节数

MiB = byte / 1024 / 1024

常见：

数组	大约内存
int a[1000000]	4 MB
long long a[1000000]	8 MB
double a[1000000]	8 MB
int dp[5000][5000]	约 100 MB
long long dp[5000][5000]	约 200 MB

10. 对齐和结构体大小

结构体可能因为对齐产生填充字节：

```
struct A {
    char c;
    int x;
};
```

常见情况下不是 5 byte，而可能是 8 byte，因为 int 需要按 4 字节对齐。

优化顺序：

```
struct B {
    int x;
    char c;
};
```

但结构体最终大小仍可能向最大对齐倍数补齐。

竞赛口令：

大规模结构体数组，要关注字段顺序和 `sizeof`。

普通算法题不为了省几个字节过度折腾。

11. 字符编码长度

编码	特点
ASCII	0..127, 英文字符常见 1 byte
UTF-8	英文 1 byte, 中文通常 3 byte
UTF-16	常见字符 2 byte, 部分字符 4 byte

C++ `string::size()` 返回字节数，不是中文字符数。若题面有中文字符串长度，通常会明确处理规则；没明确时，不要擅自按 UTF-8 字符切。

12. BCD 和固定小数

BCD：每个十进制数字用 4 bit 表示。

十进制 59 的 BCD：0101 1001

普通二进制 59：001111011

固定小数：

把金额扩大 100 倍存整数，例如 12.34 元 -> 1234 分。

适用：

- 钱、积分、百分比等需要避免浮点误差的题。
- 题面给保留两位小数且只做加减乘整数时。

SIGN-CALC-01 高等数学、微积分与数值方法

模块编号：SIGN-CALC-01

模块名称：微积分、导数积分、梯度和数值近似速查

标签：签到题、高等数学、微积分、导数、积分、梯度、牛顿法、数值积分、C++17

一句话用途：当题目把微积分概念包装成公式模拟题时，用本模块查导数、积分、梯度、数值近似和迭代规则。

题面触发词：导数、偏导、梯度、极值、凸函数、积分、面积、牛顿迭代、梯度下降、差分近似。

什么时候用：

- 题目给出明确函数和公式，要求按若干步迭代。
- 题目要求用导数判断单调、极值或用数值积分近似面积。
- 机器学习题里出现损失函数、梯度、学习率。

不要什么时候用：

- 方程求根已经给出一般一元函数，优先 SIM-07 的二分/牛顿模板。
- 题目要求严格符号推导，本卷只提供常见公式和数值模拟。
- 多变量优化规模大，不要自己写复杂优化器，按题面规则模拟。

复杂度：

- 单点导数公式：O(1)。
- 数值积分：O(n)。
- 迭代法：O(iter * eval)。

依赖的标准容器: `vector<double>`、`cmath`、`iostream`。

输入如何整理:

把函数参数和迭代次数读清楚。

若题目给学习率 `lr`, 每次更新通常是 `x -= lr * grad`。

若题目给误差 `eps`, 循环要有最大迭代次数防死循环。

接口:

`finite_diff(f,x)` -> 中心差分近似导数。

`trapezoid(f,l,r,n)` -> 梯形积分。

`simpson(f,l,r,n_even)` -> Simpson 积分。

`gradient_descent_step(x,grad,lr)` -> 一步梯度下降。

常见坑:

- 三角函数 `sin/cos/tan` 参数是弧度。
- 数值积分的 `n` 越大越准, 但太大可能超时。
- 牛顿法遇到导数接近 0 要停止或切换二分。
- 梯度下降是减梯度, 梯度上升才是加梯度。

暴力/部分替代:

- 不会解析求导时, 用中心差分近似。
- 不会积分公式时, 用梯形积分。
- 不会最优解闭式公式时, 按题面迭代固定次数。

1. 常见导数表

函数	导数
C	0
x^n	$n \cdot x^{n-1}$
$1/x$	$-1/x^2$
$\sqrt{x}$	$1/(2\sqrt{x})$
e^x	e^x
a^x	$a^x \ln a$
$\ln x$	$1/x$
$\log_a x$	$1/(x \ln a)$
$\sin x$	$\cos x$
$\cos x$	$-\sin x$
$\tan x$	$1/\cos^2 x$
$\text{sigmoid}(x)$	$s \cdot (1-s)$

2. 求导规则

$$(f+g)' = f' + g'$$

$$(f \cdot g)' = f'g + fg'$$

$$(f/g)' = (f'g - fg') / g^2$$

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$

偏导和梯度:

$$f(x,y) = x^2 + 3xy + y$$

$$df/dx = 2x + 3y$$

$$df/dy = 3x + 1$$

$$\text{gradient} = (df/dx, df/dy)$$

3. 数值方法短代码

```
double finite_diff(double (*f)(double), double x) {  
    const double h = 1e-6;  
    return (f(x + h) - f(x - h)) / (2.0 * h);  
}
```

```
double trapezoid(double (*f)(double), double l, double r, int n) {  
    double h = (r - l) / n;
```

```

double ans = (f(l) + f(r)) * 0.5;
for (int i = 1; i < n; i++) ans += f(l + h * i);
return ans * h;
}

double simpson(double (*f)(double), double l, double r, int n) {
    if (n % 2) n++;
    double h = (r - l) / n;
    double ans = f(l) + f(r);
    for (int i = 1; i < n; i++) {
        ans += f(l + h * i) * (i % 2 ? 4.0 : 2.0);
    }
    return ans * h / 3.0;
}

```

4. 常见原函数和积分

函数	一个原函数
x^n	$x^{(n+1)/(n+1)}, n \neq -1$
$1/x$	$\ln x $
e^x	e^x
$\sin x$	$-\cos x$
$\cos x$	$\sin x$
$1/(1+x^2)$	$\arctan x$

5. 优化和机器学习常见式

场景	公式
平方损失	$L = (\text{pred} - y)^2$
MSE	$\text{sum}((\text{pred} - y)^2) / n$
一维梯度下降	$x = x - \text{lr} * \text{grad}(x)$
线性回归预测	$\text{pred} = w * x + b$
logistic	$\text{sigmoid}(z) = 1 / (1 + \exp(-z))$
二分类交叉熵	$-y \ln p - (1 - y) \ln(1 - p)$

SIGN-COMP-01 复杂度、数据规模与工程估算

模块编号: SIGN-COMP-01

模块名称: 复杂度、数据规模、内存和工程数量级估算

标签: 签到题、复杂度、内存估算、数量级、工程估算、C++17

一句话用途: 把数据范围快速翻译成可用复杂度、内存占用和实现风险, 防止签到题和简单算法题因为估算错误丢分。

题面触发词: 数据范围、内存限制、时间限制、操作次数、数组大小、矩阵大小、提交限制、复杂度。

什么时候用:

- 读题后要判断暴力能不能过。
- 题目问某算法复杂度或内存占用。
- 需要估算 `int` 数组、`long long` 数组、二维矩阵占用。

不要什么时候用:

- 精确性能受平台影响, 本卷只做数量级判断。
- 卡常极限题要结合实际编译器和数据分布。

复杂度: 估算本身 $O(1)$ 。

依赖的标准容器: 无固定依赖; 常见估算对象是数组、vector、图邻接表。

输入如何整理:

圈 n, m, q, T 。
 看是否给多测总和。
 估算状态数 * 每状态转移。
 估算数组元素个数 * 单元素字节。

接口：

$ops \leq 1e8$ 一般可试。
 $memory_bytes = count * sizeof(type)$ 。
 $graph\ memory \sim edges * edge_record_size$ 。

常见坑：

- 多测试没有总和时，不能只看单测。
- `vector<vector<int>>` 有额外开销，静态数组更容易估算。
- `bool` 数组通常 1 byte, `bitset` 才压位。
- 递归深度大可能栈爆。

暴力/部分替代：

- 正解不会时，根据数据范围分档：小数据暴力，大数据合法兜底。
- 内存不够时改滚动数组、压位、邻接表。

1. 时间预算表

最大操作量	直觉
1e6	很安全
1e7	安全
1e8	C++ 可试
1e9	通常危险
1e10	基本不行

数据范围：

范围	可用复杂度
$n \leq 10$	$n!$
$n \leq 20$	$2^n * n$
$n \leq 40$	折半 $2^{(n/2)}$
$n \leq 300$	n^3
$n \leq 3000$	n^2
$n \leq 2e5$	$n \log n$
$n \leq 1e6$	n 或较小常数 $n \log n$

2. 内存估算

类型	常见字节
<code>char</code>	1
<code>bool</code>	1
<code>int</code>	4
<code>long long</code>	8
<code>double</code>	8
<code>pair<int,int></code>	通常 8
<code>pair<long long,int></code>	可能 16

估算例子：

`int a[1000000]` -> 约 4 MB
`long long dp[5000][5000]` -> $5000 * 5000 * 8 = 200$ MB, 危险
`int dist[1000][1000]` -> 4 MB

3. 图存储估算

邻接表边:

```
struct Edge {
    int to;
    int w;
    int next;
}; // 通常 12 byte
```

无向图要存两条边。

$m=2e5$ 无向边 $\rightarrow$ $4e5$ 条 Edge $\rightarrow$ 约 4.8 MB

4. 常见优化方向

超时来源	优化方向
三重循环	前缀和、排序、DP 优化
枚举所有对	排序 + 双指针 / 哈希
重复 DFS	记忆化
区间反复求和	前缀和 / 树状数组 / 线段树
全源最短路太大	多次 Dijkstra 或只求需要点
二维 DP 内存大	滚动数组
字符串反复截取	指针/下标, 避免复制

SIGN-CS-01 计算机常识、编码、文件和网络

模块编号: SIGN-CS-01

模块名称: 计算机基础常识: 存储单位、编码、文件大小、网络和格式

标签: 签到题、计算机常识、bit、byte、ASCII、UTF-8、BMP、音频、视频、网络、C++17

一句话用途: 遇到文件大小、图片音视频、进制编码、IP、带宽、ASCII/UTF-8 等题时, 用本模块先统一单位再计算。

题面触发词: bit、byte、KB、KiB、BMP、RGB、RGBA、采样率、码率、ASCII、UTF-8、IPv4、CIDR、Base64、URL 编码。

什么时候用:

- 题目问内存、文件大小、传输时间、编码转换。
- 输入是图片/音频/视频参数, 要求估算容量。
- 题目考二进制、十六进制、补码、校验和。

不要什么时候用:

- 复杂文件格式解析, 优先 SIM-04。
- 真正图像算法如卷积、滤波, 优先按矩阵/模拟题处理。
- 网络协议细节题若题面另给规则, 以题面为准。

复杂度:

- 单个换算 $O(1)$ 。
- 字符串编码扫描 $O(\text{len})$ 。
- IP/CIDR 解析 $O(\text{len})$ 。

依赖的标准容器: string、vector<int>、sstream、iomanip。

输入如何整理:

先看单位: bit 还是 byte, KB 还是 KiB, bps 还是 B/s。

图片先看每像素位数和行对齐。

网络传输先把带宽转成 byte/s。

接口:

bmp_size(width,height,bits) $\rightarrow$ BMP 像素数据字节数, 按 4 字节行对齐。

ipv4_to_int(s) $\rightarrow$ IPv4 转 32 位整数。

base_convert(s,base) $\rightarrow$ 小范围进制转十进制。

常见坑:

- 1 byte = 8 bit.
- Mbps 是百万 bit/s, 不是 MB/s.
- BMP 24 位图每像素 3 byte, 但每行要补到 4 的倍数.
- UTF-8 中文通常占 3 byte, 不等于字符个数.
- 十六进制字符 A-F 可大小写.

暴力/部分替代:

- 编码规则复杂时, 先处理 ASCII 和题面出现的字符.
- BMP 若忘记行对齐, 先交 $w \times h \times \text{bytes}$ 可能拿部分分.
- IP/CIDR 不熟时, 小范围直接字符串分段比较.

1. 单位表

单位	含义
bit	位
byte / B	字节, 1B=8bit
KB	常见十进制 1000B, 看题面
KiB	1024B
MB	1000^2B, 看题面
MiB	1024^2B
bps	bit per second
B/s	byte per second

下载时间:

```
seconds = file_bytes * 8 / bandwidth_bps
```

2. BMP 和媒体大小

```
long long bmp_pixel_bytes(long long width, long long height, int bits_per_pixel) {
    long long row_bits = width * bits_per_pixel;
    long long row_bytes = ((row_bits + 31) / 32) * 4; // BMP 行 4 字节对齐
    return row_bytes * height;
}

long long audio_bytes(long long seconds, long long sample_rate, int bits_per_sample, int channels) {
    return seconds * sample_rate * bits_per_sample / 8 * channels;
}

long long video_bytes_by_bitrate(long long seconds, long long bitrate_bps) {
    return seconds * bitrate_bps / 8;
}
```

常见图像:

格式	每像素
灰度 8 bit	1 byte
RGB 24 bit	3 byte
RGBA 32 bit	4 byte
BMP 24 bit	3 byte + 行对齐

3. 编码和进制

```
int hex_value(char c) {
    if ('0' <= c && c <= '9') return c - '0';
    if ('a' <= c && c <= 'f') return c - 'a' + 10;
    if ('A' <= c && c <= 'F') return c - 'A' + 10;
    return -1;
}
```

```

long long to_decimal(const string &s, int base) {
    long long ans = 0;
    for (char c : s) ans = ans * base + hex_value(c);
    return ans;
}

int utf8_byte_count(unsigned char c) {
    if ((c & 0x80) == 0) return 1;
    if ((c & 0xE0) == 0xC0) return 2;
    if ((c & 0xF0) == 0xE0) return 3;
    if ((c & 0xF8) == 0xF0) return 4;
    return 1;
}

```

4. IPv4 / CIDR

```

unsigned int ipv4_to_uint(const string &s) {
    unsigned int ans = 0, cur = 0;
    for (char c : s) {
        if (c == '.') {
            ans = (ans << 8) | cur;
            cur = 0;
        } else {
            cur = cur * 10 + (c - '0');
        }
    }
    return (ans << 8) | cur;
}

bool same_cidr(const string &a, const string &b, int prefix) {
    unsigned int x = ipv4_to_uint(a), y = ipv4_to_uint(b);
    if (prefix == 0) return true;
    unsigned int mask = 0xffffffffu << (32 - prefix);
    return (x & mask) == (y & mask);
}

```

SIGN-LA-01 线性代数、向量与小矩阵

模块编号: SIGN-LA-01

模块名称: 线性代数常识: 向量、矩阵、距离、投影和小规模变换

标签: 签到题、线性代数、向量、矩阵、距离、点积、叉积、Markov、C++17

一句话用途: 遇到向量、矩阵、坐标变换、小规模线性代数或机器学习特征计算时, 用本模块快速查公式。

题面触发词: 向量、矩阵、点积、叉积、距离、投影、旋转、转移矩阵、特征、线性组合。

什么时候用:

- 坐标、几何、机器学习特征题需要向量计算。
- 题目给小矩阵, 要求乘法、转置、行列式、逆矩阵或转移若干步。
- 题目出现 Markov 转移、状态概率、二维旋转。

不要什么时候用:

- 大规模线性方程组求解, 优先 SIM-07 高斯消元。
- 复杂矩阵快速幂, 优先 MATH-05。
- 高阶线代证明题, 本卷只服务计算和模拟。

复杂度:

- 向量距离/点积: $O(d)$ 。
- 矩阵乘法: $O(n*m*k)$ 。
- 2x2 行列式/逆: $O(1)$ 。

依赖的标准容器: `vector<double>`、静态二维数组、`cmath`、`iostream`。

输入如何整理:

矩阵 A 的尺寸是 `rows x cols`。

A*B 能乘的条件: `A.cols == B.rows`。

向量维度必须一致。

接口:

`dot(a,b,d)` -> 点积。

`norm2(a,d)` -> 平方范数。

`dist2(a,b,d)` -> 平方欧氏距离。

`cross2(ax,ay,bx,by)` -> 二维叉积。

`mat_mul(A,B)` -> 小矩阵乘法。

常见坑:

- 矩阵乘法不满足交换律, $A*B$ 和 $B*A$ 通常不同。
- 欧氏距离比较大小时可比较平方距离, 少开根。
- cosine 相似度分母为 0 要特判。
- 旋转角必须是弧度。

暴力/部分替代:

- 小矩阵直接三重循环。
- 多步转移次数很小直接重复乘; 次数很大再用矩阵快速幂。
- 维度很低时直接展开公式。

1. 向量公式

名称	公式
点积	$a \cdot b = \sum a_i b_i$
范数	$\ a\ = \sqrt{a \cdot a}$
欧氏距离	$\sqrt{\sum (a_i - b_i)^2}$
曼哈顿距离	$\sum a_i - b_i $
cosine 相似度	$(a \cdot b) / (\ a\ \ b\)$
二维叉积	$a_x b_y - a_y b_x$
三角形有向面积	$\text{cross}(B-A, C-A) / 2$
投影长度	$(a \cdot b) / \ b\ $

```
double dot_product(const double a[], const double b[], int d) {
    double s = 0;
    for (int i = 1; i <= d; i++) s += a[i] * b[i];
    return s;
}
```

```
double dist2(const double a[], const double b[], int d) {
    double s = 0;
    for (int i = 1; i <= d; i++) {
        double t = a[i] - b[i];
        s += t * t;
    }
    return s;
}
```

```
double cross2(double ax, double ay, double bx, double by) {
    return ax * by - ay * bx;
}
```

2. 小矩阵公式

名称	公式
2x2 行列式	$ad - bc$
2x2 逆矩阵	$1/\det * \begin{bmatrix} d, -b \\ -c, a \end{bmatrix}$

名称	公式
trace	主对角线和
对称矩阵	$A[i][j] == A[j][i]$
单位矩阵	对角线 1, 其余 0
转置	$B[j][i] = A[i][j]$

```
const int MAXD = 55;
double A[MAXD][MAXD], B[MAXD][MAXD], C[MAXD][MAXD];

void mat_mul(int n, int m, int k) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            C[i][j] = 0;
            for (int t = 1; t <= m; t++) {
                C[i][j] += A[i][t] * B[t][j];
            }
        }
    }
}
```

3. 机器学习里常见线代

场景	计算
线性模型打分	$score = w \cdot x + b$
全连接层	$y[j] = b[j] + \sum_i x[i] * W[i][j]$
kNN 距离	常用欧氏距离或曼哈顿距离
文本相似度	cosine 或 Jaccard
Markov 一步转移	$next = cur * P$
PCA 低维投影	$z = x \cdot direction$

SIGN-LOGIC-01 离散逻辑、布尔代数与自动机

模块编号: SIGN-LOGIC-01

模块名称: 离散数学常识: 逻辑、集合、关系、自动机和形式语言

标签: 签到题、离散数学、逻辑、集合、关系、自动机、正则、状态机、C++17

一句话用途: 遇到命题逻辑、集合关系、布尔表达式、状态机、自动机和形式语言概念题时, 用本模块快速查规则。

题面触发词: 命题、真值表、与或非、蕴含、集合、关系、等价关系、偏序、DFA、NFA、正则表达式。

什么时候用:

- 题目要求判断逻辑表达式真假或等价。
- 题目给状态转移表, 要求模拟自动机。
- 题目考集合、关系、函数、映射的基础定义。

不要什么时候用:

- 复杂图论题回到图论卷。
- 正则匹配完整实现可用 DP/自动机, 按题面规模选择。
- 编译器/解释器题优先 SIM-05。

复杂度:

- 真值表 k 个变量是 $O(2^k)$ 。
- DFA 模拟 $O(len)$ 。
- 集合操作按排序/哈希实现通常 $O(n \log n)$ 或均摊 $O(n)$ 。

依赖的标准容器: vector、set、map、string。

输入如何整理:

逻辑题先列变量。

自动机题先记录起始状态、接受状态和转移表。

集合题先去重。

接口：

truth table -> 枚举 mask。

DFA simulate -> $state = trans[state][char]$ 。

set relation -> 检查自反、对称、传递。

常见坑：

- $A \rightarrow B$ 只有 A 真 B 假时为假。
- 德摩根: $\neg(A \text{ and } B) = \neg A \text{ or } \neg B$ 。
- 空集是任意集合的子集。
- DFA 每个状态每个字符只有一个转移; NFA 可以有多个。

暴力/部分替代：

- 变量少时真值表枚举最稳。
- 自动机状态少时直接二维表模拟。
- 关系性质不会抽象时，三重循环检查传递性。

1. 逻辑公式

公式	等价
$A \rightarrow B$	$\neg A \text{ or } B$
$A \leftrightarrow B$	$(A \text{ and } B) \text{ or } (\neg A \text{ and } \neg B)$
$\neg(A \text{ and } B)$	$\neg A \text{ or } \neg B$
$\neg(A \text{ or } B)$	$\neg A \text{ and } \neg B$
双重否定	$\neg\neg A = A$
分配律	$A \text{ and } (B \text{ or } C) = (A \text{ and } B) \text{ or } (A \text{ and } C)$

真值表命令：

```
for (int mask = 0; mask < (1 << k); mask++) {  
    bool a = mask & 1;  
    bool b = mask & 2;  
    bool implication = (!a) || b;  
}
```

2. 集合和关系

概念	定义
子集	A 中每个元素都在 B 中
真子集	子集且不相等
交集	同时属于
并集	至少属于一个
差集	属于 A 不属于 B
自反	对所有 a , aRa
对称	aRb 则 bRa
反对称	aRb 且 bRa 则 $a=b$
传递	aRb 且 bRc 则 aRc
等价关系	自反、对称、传递
偏序	自反、反对称、传递

3. 自动机

DFA 五元组：

状态集合 Q

字母表 Σ

转移函数 δ

起始状态 $start$

接受状态集合 F

模拟：

```
int simulate_dfa(const vector<array<int, 26>> &go, int start, const string &s) {
    int state = start;
    for (char c : s) state = go[state][c - 'a'];
    return state;
}
```

4. 正则表达式常识

符号	含义
a	字符 a
.	任意一个字符，按题面
*	重复 0 次或多次
+	重复 1 次或多次
?	0 次或 1 次
	或
()	分组

SIGN-MARKOV-01 马尔可夫性质、Markov 链与状态转移

模块编号：SIGN-MARKOV-01

模块名称：马尔可夫性质：Markov 链、转移矩阵、平稳分布、吸收状态、HMM 和 MDP

标签：签到题、概率、马尔可夫性质、Markov 链、转移矩阵、平稳分布、吸收链、HMM、MDP、C++17

一句话用途：当题目说“下一步只和当前状态有关”、给状态转移概率矩阵、要求若干步后的分布或长期比例时，用本模块按 Markov 链处理。

题面触发词：

- 马尔可夫性质、Markov property、无后效性、记忆无关。
- 状态转移矩阵、一步转移概率、 $P[i][j]$ 。
- 给初始分布，求第 k 步在各状态的概率。
- 长期稳定概率、平稳分布、steady state。
- 吸收状态、最终到达某状态的概率。
- HMM、Viterbi、MDP、强化学习。

什么时候用：

- 系统未来只由当前状态决定，不需要知道更早历史。
- 题目给的是概率状态机，而不是确定性自动机。
- 需要重复乘转移矩阵或迭代分布。
- AI/RL 题中状态、动作、转移、奖励明确给出。

不要什么时候用：

- 下一步依赖最近两步或更长历史时，原状态不满足 Markov 性质；要把“上一状态/上一步动作”等历史信息并入状态。
- 转移概率随时间改变时，不是齐次 Markov 链；要按每一步自己的矩阵乘。
- 状态数很大且步数很大时，不能直接 $O(k \cdot n^2)$ ，考虑矩阵快速幂或稀疏图。
- 题目要求最短路/最优策略而非概率演化时，可能是图论或 DP。

复杂度：

- 分布迭代 k 步： $O(k \cdot n^2)$ 。
- 转移矩阵快速幂： $O(n^3 \log k)$ 。
- 稀疏转移每步： $O(k \cdot m)$ 。
- 平稳分布迭代： $O(\text{iter} \cdot n^2)$ 。
- 吸收概率可用方程组，或按迭代近似。

数据范围参考：

- $n \leq 50$ 且 k 很大：矩阵快速幂。
- $n \leq 1000$ 但边很少：稀疏转移迭代。
- 只问几十步：直接分布迭代最简单。
- 要长期比例且链收敛：迭代到稳定或解线性方程。

依赖的标准容器：

- 静态数组 `double P[MAXN][MAXN]`：转移矩阵，1-index。
- 静态数组 `double dist[MAXN]`：当前分布。
- `vector<pair<int,double>> g[MAXN]`：稀疏转移。
- `iomanip`：概率输出。

输入如何整理：

1. 状态编号统一 $1..n$ 。
2. $P[i][j]$ 表示从状态 i 到状态 j 的概率。
3. 每行概率和通常为 1；若题面允许误差，用 EPS 检查。
4. 初始分布 $dist[i]$ 也应和为 1。

接口：

`iterate_distribution(n,k,dist,P)` -> 直接做 k 步分布。
`matrix_power_distribution(n,k,dist,P)` -> 矩阵快速幂做 k 步分布。
`stationary_iter(n,dist,P,iter)` -> 迭代近似平稳分布。
`is_markov_state_enough()` -> 若未来还依赖历史，升维状态。

常见坑：

- 把 $P[i][j]$ 当成 $P[j][i]$ ，行列方向反了。
- 初始分布不是概率分布，和不为 1。
- 每行转移概率和不为 1，却没有按题面解释成权重。
- 题目实际依赖上一步动作或上一个状态，却只把当前位置当状态。
- 矩阵快速幂中行向量/列向量约定混乱。
- 平稳分布不一定存在唯一极限，周期链可能震荡。
- 浮点输出不要用 `==` 比较概率。

暴力/部分替代：

- k 小时直接一步一步模拟分布。
- 状态数小但历史依赖时，把最近历史并入状态，例如（当前点，上一步方向）。
- 不会平稳分布精确解时，迭代 1000 到 10000 轮拿近似分。
- 吸收概率不会列方程时，迭代很多步近似最终分布。
- HMM 不会 Viterbi 时，小规模枚举隐状态序列拿部分分。

1. Markov 性质到底是什么

核心公式：

$$P(X_{t+1}=j \mid X_t=i, X_{t-1}, \dots, X_0) = P(X_{t+1}=j \mid X_t=i)$$

中文口令：

未来只看现在，不看过去。

这和 DP 的“无后效性”很像：

DP 语境	Markov 语境
状态包含决定未来的全部信息	当前状态包含下一步概率所需全部信息
历史不影响后续转移	更早历史不影响下一步概率
有后效性就升维	不满足 Markov 就把关键历史并入状态

例子：

不能连续向下走两步：

只用位置 (i,j) 不够，因为下一步能否向下取决于上一步方向。

升维为 $(i,j,last_dir)$ 后，就恢复 Markov/无后效性。

2. 转移矩阵

若有 n 个状态， $P[i][j]$ 表示从 i 到 j 的概率。

$$dist_next[j] = \sum_i dist[i] * P[i][j]$$

矩阵写法：

$\text{dist_after_k} = \text{dist_initial} * P^k$

注意:

- 本卷默认行向量分布, 所以是 $\text{dist} * P$ 。
- 有些教材用列向量, 会写 $P * \text{dist}$, 不要混淆。
- 竞赛题通常直接给 $P[i][j]$, 按题意。

3. 直接迭代模板

适合 k 不大, 或者 n 较大但转移稀疏。

```
const int MAXN = 105;
double P[MAXN][MAXN], distv[MAXN], ndist[MAXN];

void iterate_distribution(int n, long long k) {
    for (long long step = 1; step <= k; step++) {
        for (int j = 1; j <= n; j++) ndist[j] = 0;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                ndist[j] += distv[i] * P[i][j];
            }
        }
        for (int j = 1; j <= n; j++) distv[j] = ndist[j];
    }
}
```

4. 矩阵快速幂模板

适合 k 很大、 n 不大。

```
const int MAXN = 105;
int N;
double A[MAXN][MAXN], R[MAXN][MAXN], T[MAXN][MAXN];

void mat_mul(double X[][MAXN], double Y[][MAXN], double Z[][MAXN]) {
    static double C[MAXN][MAXN];
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            C[i][j] = 0;
            for (int k = 1; k <= N; k++) C[i][j] += X[i][k] * Y[k][j];
        }
    }
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) Z[i][j] = C[i][j];
    }
}

void mat_pow(long long e) {
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) R[i][j] = (i == j);
    }
    while (e > 0) {
        if (e & 1) mat_mul(R, A, R);
        mat_mul(A, A, A);
        e >>= 1;
    }
}
```

得到 P^k 后:

```
for (int j = 1; j <= n; j++) {
    ans[j] = 0;
    for (int i = 1; i <= n; i++) ans[j] += dist[i] * R[i][j];
}
```

5. 平稳分布

平稳分布 π 满足：

```
 $\pi = \pi * P$   
 $\sum \pi[i] = 1$ 
```

直觉：

- 如果链满足一定连通/非周期条件，反复转移会趋向一个稳定分布。
- 题目若只要求近似，直接迭代很多轮通常够用。

```
void stationary_iter(int n, int iter) {  
    for (int i = 1; i <= n; i++) distv[i] = 1.0 / n;  
    for (int step = 1; step <= iter; step++) {  
        for (int j = 1; j <= n; j++) ndist[j] = 0;  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= n; j++) ndist[j] += distv[i] * P[i][j];  
        }  
        for (int j = 1; j <= n; j++) distv[j] = ndist[j];  
    }  
}
```

6. 吸收状态

吸收状态：

$P[x][x] = 1$ ，且不会离开 x 。

常见问题：

- 最终被哪个吸收状态吸收的概率。
- 到吸收状态的期望步数。

处理方式：

问题	方法
小数据近似	迭代很多步
精确吸收概率	列线性方程组，用 SIM-07 高斯
期望步数	$E[u] = 1 + \sum P[u][v]E[v]$ ，吸收态 $E=0$

7. HMM、MDP、强化学习的关系

名称	核心
Markov 链	只有状态转移
HMM	隐状态 Markov，另有观测概率
MDP	状态 + 动作 + 转移概率 + 奖励
Q-learning	学 $Q[state][action]$
Viterbi	HMM 中求最可能隐状态路径

HMM 的两个概率：

```
transition:  $P(\text{hidden}_t \rightarrow \text{hidden}_{t+1})$   
emission:  $P(\text{observation}_t | \text{hidden}_t)$ 
```

MDP 的 Markov 性质：

下一状态和奖励只依赖当前状态与当前动作，不依赖更早历史。

8. 完整可运行模板

支持三种模式：

- step: 直接迭代 k 步。
- power: 矩阵快速幂求 k 步。
- stationary: 迭代近似平稳分布。

```

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 105;
int N;
double P[MAXN][MAXN], A[MAXN][MAXN], R[MAXN][MAXN];
double distv[MAXN], ansv[MAXN], ndist[MAXN];

void mat_mul(double X[][MAXN], double Y[][MAXN], double Z[][MAXN]) {
    static double C[MAXN][MAXN];
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            C[i][j] = 0;
            for (int k = 1; k <= N; k++) C[i][j] += X[i][k] * Y[k][j];
        }
    }
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) Z[i][j] = C[i][j];
    }
}

void mat_pow(long long e) {
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) R[i][j] = (i == j ? 1.0 : 0.0);
    }
    while (e > 0) {
        if (e & 1) mat_mul(R, A, R);
        mat_mul(A, A, A);
        e >>= 1;
    }
}

void print_dist(double d[]) {
    cout << fixed << setprecision(6);
    for (int i = 1; i <= N; i++) {
        if (i > 1) cout << ' ';
        cout << d[i];
    }
    cout << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string mode;
    long long k;
    cin >> mode >> N >> k;
    for (int i = 1; i <= N; i++) cin >> distv[i];
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            cin >> P[i][j];
            A[i][j] = P[i][j];
        }
    }

    if (mode == "step") {
        for (long long step = 1; step <= k; step++) {
            for (int j = 1; j <= N; j++) ndist[j] = 0;
            for (int i = 1; i <= N; i++) {
                for (int j = 1; j <= N; j++) ndist[j] += distv[i] * P[i][j];
            }
            for (int j = 1; j <= N; j++) distv[j] = ndist[j];
        }
    }
}

```

```

    }
    print_dist(distv);
} else if (mode == "power") {
    mat_pow(k);
    for (int j = 1; j <= N; j++) {
        ansv[j] = 0;
        for (int i = 1; i <= N; i++) ansv[j] += distv[i] * R[i][j];
    }
    print_dist(ansv);
} else if (mode == "stationary") {
    for (int i = 1; i <= N; i++) distv[i] = 1.0 / N;
    for (long long step = 1; step <= k; step++) {
        for (int j = 1; j <= N; j++) ndist[j] = 0;
        for (int i = 1; i <= N; i++) {
            for (int j = 1; j <= N; j++) ndist[j] += distv[i] * P[i][j];
        }
        for (int j = 1; j <= N; j++) distv[j] = ndist[j];
    }
    print_dist(distv);
}
return 0;
}

```

9. 最小测试样例

```

step 2 2
1 0
0.5 0.5
0.2 0.8
=> 0.350000 0.650000

```

```

power 2 10
1 0
0.5 0.5
0.2 0.8
=> 0.285719 0.714281

```

```

stationary 2 100
0 0
0.5 0.5
0.2 0.8
=> 0.285714 0.714286

```

10. 考场判断清单

- 当前状态是否包含影响未来的全部信息？
- 转移概率每行是否和为 1？
- 分布是行向量还是列向量？
- k 大不大？大则考虑矩阵快速幂。
- 是否有吸收态？
- 是否要求最优策略？若是 MDP/强化学习，不只是普通 Markov 链。

SIGN-MATH-01 初等数学、单位与几何公式

模块编号: SIGN-MATH-01

模块名称: 初等数学、单位换算、常见几何和公式签到题

标签: 签到题、初等数学、几何、单位换算、数列、取整、百分比、C++17

一句话用途: 把最容易出成签到题的数学公式集中放在一页，避免现场忘记三角形面积、取整、增长率、单位换算等细节。

题面触发词: 三角形面积、圆、扇形、速度、折扣、利润率、百分比、等差数列、等比数列、四舍五入、单位换算。

什么时候用：

- 题目核心是公式计算或简单分类讨论。
- 数据范围很小，但单位和输出格式容易错。
- 题目问面积、体积、增长率、折扣、利息、页码、植树、鸡兔同笼等。

不要什么时候用：

- 坐标几何有大量点线关系，优先 MATHREF-06 或几何模块。
- 方程组、数值求根、矩阵解法，优先 SIM-07 。
- 组合计数很复杂，优先第 8 卷数学参考。

复杂度：大多 $O(1)$ ；排序统计类 $O(n \log n)$ ；逐项模拟类 $O(n)$ 。

依赖的标准容器：vector<double>、vector<long long>、string、algorithm、iomanip。

输入如何整理：

先统一单位，再套公式。

长度、面积、体积不要混用单位。

百分数输入如果是 20，先确认是 20% 还是 20 倍。

接口：

ceil_div(a,b) -> 正整数向上取整。

triangle_area_heron(a,b,c) -> 三边三角形面积。

deg_to_rad(x) / rad_to_deg(x) -> 角度弧度转换。

arith_sum(a1,d,n) / geom_sum(a1,r,n) -> 数列求和。

常见坑：

- pow(10, k) 返回浮点，不适合精确整数幂，整数幂自己循环或快速幂。
- round() 返回最接近整数，但输出保留小数应用 fixed << setprecision(k)。
- 海伦公式中 $s*(s-a)*(s-b)*(s-c)$ 可能因误差略为负，开根前可 $\max(0.0, x)$ 。
- a/b 如果都是整数会整除，需要写 $1.0*a/b$ 。

暴力/部分替代：

- 几何公式忘记时，坐标多边形面积可用三角剖分或叉积。
- 数列公式忘记时，n 小可循环累加。
- 复杂分段计费不会化简时，按题面逐段模拟。

1. 常用公式清单

题型	公式/规则	坑
正整数向上取整	$(a+b-1)/b$	只适用于 $a, b > 0$
负数取模转非负	$(x \% \text{mod} + \text{mod}) \% \text{mod}$	C++ 负数 % 仍可能负
百分比增长	$(\text{new} - \text{old}) / \text{old} * 100\%$	old=0 要特判
利润率	$\text{profit} / \text{cost}$	题面可能用售价作分母
折扣	$\text{price} * \text{discount} / 10$ 或 $\text{price} * \text{rate}$	九折是 0.9
单利	$P * (1 + r * t)$	r 是每期利率
复利	$P * \text{pow}(1 + r, t)$	注意年/月单位
等差求和	$n * (a_1 + a_n) / 2$	乘法用 long long
等比求和	$a_1 * (1 - r^n) / (1 - r)$	r=1 特判
平方和	$n(n+1)(2n+1)/6$	防溢出
立方和	$[n(n+1)/2]^2$	防溢出
圆面积	$\pi * r * r$	角度无关
圆周长	$2 * \pi * r$	直径是 2r
扇形面积	$\text{theta} / 360 * \pi * r * r$	theta 若是弧度则 $0.5 * r * r * \text{theta}$
弧长	$\text{theta} / 360 * 2 * \pi * r$	弧度时 $r * \text{theta}$
三角形合法	$a > b + c \ \&\& \ a + c > b \ \&\& \ b + c > a$	先排序更简单
海伦公式	$\text{sqrt}(s(s-a)(s-b)(s-c))$	$s = (a+b+c)/2$
梯形面积	$(\text{上底} + \text{下底}) * \text{高} / 2$	类型转 double
球体体积	$4/3 * \pi * r^3$	写 4.0/3
圆柱体积	$\pi * r * r * h$	单位一致
圆锥体积	$\pi * r * r * h / 3$	写 /3.0

2. 高频小函数

```
using ll = long long;
const double PI = acos(-1.0);

ll ceil_div_pos(ll a, ll b) {
    return (a + b - 1) / b;
}

double deg_to_rad(double deg) {
    return deg * PI / 180.0;
}

double rad_to_deg(double rad) {
    return rad * 180.0 / PI;
}

bool triangle_ok(double a, double b, double c) {
    return a + b > c && a + c > b && b + c > a;
}

double triangle_area_heron(double a, double b, double c) {
    if (!triangle_ok(a, b, c)) return -1.0;
    double s = (a + b + c) / 2.0;
    return sqrt(max(0.0, s * (s - a) * (s - b) * (s - c)));
}

ll arith_sum(ll a1, ll d, ll n) {
    return n * (2 * a1 + (n - 1) * d) / 2;
}
```

3. 签到题模型补充

模型	规则
植树问题	不成环：棵数 = 段数 + 1；成环：棵数 = 段数
页码数字统计	小数据直接从 1 到 n 转字符串统计
鸡兔同笼	$x+y=n$, $2x+4y=m$
年龄问题	设当前年龄，按年份差列方程
工程问题	总工作量设为 1，效率相加
相遇追及	相遇：相对速度相加；追及：速度相减
阶梯计价	按区间逐段扣减最稳

SIGN-MEDIA-02 多媒体、文件格式与压缩估算

模块编号：SIGN-MEDIA-02

模块名称：图片、音频、视频、文件格式和压缩估算

标签：签到题、多媒体、图片、音频、视频、BMP、颜色、压缩、文件大小、C++17

一句话用途：遇到图片大小、采样率、码率、颜色编码、压缩率和文件格式估算时，用本模块查公式。

题面触发词：BMP、像素、分辨率、DPI、RGB、RGBA、灰度、采样率、声道、帧率、码率、压缩率。

什么时候用：

- 题目给宽高、位深、采样率、帧率，要求文件大小。
- 题目问 RGB/HSV、alpha、颜色十六进制。
- 题目问压缩前后比例或传输时间。

不要什么时候用：

- 真正图像处理算法，如卷积、边缘检测，按矩阵模拟。
- 复杂文件格式头部结构，以题面给出的字段为准。

- 历史编码和字体渲染细节不在本卷范围内。

复杂度：公式估算 $O(1)$ ；像素逐个处理 $O(\text{width} * \text{height})$ 。

依赖的标准容器：string、vector<int>、iomanip。

输入如何整理：

图像：宽、高、每像素 bit、是否行对齐。

音频：秒数、采样率、每样本 bit、声道数。

视频：秒数、帧率、每帧大小或码率。

接口：

```
image_raw_bytes = width * height * bits_per_pixel / 8.
BMP_bytes = row_aligned_bytes * height.
audio_bytes = seconds * sample_rate * bits_per_sample/8 * channels.
video_bytes = seconds * bitrate_bps / 8.
```

常见坑：

- BMP 像素数据每行 4 字节对齐。
- 24 位 RGB 是 3 字节，不含 alpha。
- 32 位 RGBA 是 4 字节。
- DPI 是打印密度，不直接改变像素总数，除非题目用英寸换算像素。
- 码率通常已经包含压缩后每秒 bit 数。

暴力/部分替代：

- 不知道头部大小时，先算像素数据大小。
- 不知道压缩格式时，按题面给的压缩率。
- RGB 转换复杂时，先处理十六进制拆分。

1. 图片大小

图像类型	每像素
黑白 1 bit	1/8 byte
灰度 8 bit	1 byte
RGB 24 bit	3 byte
RGBA 32 bit	4 byte
16 bit 色	2 byte

BMP 行对齐：

```
row_bytes = ceil(width * bits_per_pixel / 32) * 4
total_pixel_bytes = row_bytes * height
```

2. 颜色

表示	含义
#RRGGBB	红绿蓝各 8 bit
#AARRGGBB	alpha + RGB
RGB	红绿蓝
BGR	BMP 等格式常见存储顺序
Alpha	透明度
HSV	色相、饱和度、明度

十六进制颜色拆分：

```
int hex2(char a, char b) {
    auto val = [](char c) {
        if ('0' <= c && c <= '9') return c - '0';
        if ('a' <= c && c <= 'f') return c - 'a' + 10;
        return c - 'A' + 10;
    };
};
```

```

    return val(a) * 16 + val(b);
}

```

3. 音频大小

$\text{bytes} = \text{seconds} * \text{sample_rate} * \text{bits_per_sample} / 8 * \text{channels}$

例子:

60 秒, 44100 Hz, 16 bit, 双声道:
 $60 * 44100 * 16 / 8 * 2 = 10584000 \text{ byte}$

4. 视频大小

两种常见题面:

未压缩: $\text{width} * \text{height} * \text{bytes_per_pixel} * \text{fps} * \text{seconds}$
 有码率: $\text{bitrate_bps} * \text{seconds} / 8$

5. 压缩率

问法	公式
压缩后大小	$\text{original} * \text{ratio}$
压缩节省	$\text{original} - \text{compressed}$
节省百分比	$(\text{original} - \text{compressed}) / \text{original}$
码率估算	$\text{file_bits} / \text{seconds}$

SIGN-ML-01 机器学习算法常识与小模拟

模块编号: SIGN-ML-01

模块名称: 机器学习签到题: 评估指标、常见模型和按公式模拟

标签: 签到题、机器学习、监督学习、无监督学习、分类、回归、聚类、SVM、DNN、C++17

一句话用途: AI 背景题如果只是给小数据和规则, 按本模块公式模拟, 不需要任何第三方库。

题面触发词: 训练集、测试集、标签、特征、分类、回归、聚类、SVM、DNN、softmax、precision、recall、F1。

什么时候用:

- 题目要求按给定公式算预测、指标或若干轮训练。
- 数据规模小, 可以直接二维数组或 vector 模拟。
- 题目是 Special Judge, 要求最大化某个评估指标。

不要什么时候用:

- 不要把真实机器学习库思路带进考场, 不能用第三方库。
- 大规模稀疏文本检索优先倒排索引、排序和哈希。
- 神经网络复杂反传优先第 10 卷 AI-14/15。

复杂度:

- 混淆矩阵: $O(n)$ 。
- kNN: $O(q * n * d)$ 。
- k-means 一轮: $O(n * k * d)$ 。
- 全连接层前向: $O(\text{in} * \text{out})$ 。

依赖的标准容器: vector<double>、vector<int>、map、sort、cmath。

输入如何整理:

样本表常见格式: n d, 然后每行 d 个特征和 1 个标签。

特征一般用 double, 标签一般用 int/string。

分类指标先数 TP/FP/FN/TN。

接口:

confusion -> 混淆矩阵。
 metrics_binary -> accuracy/precision/recall/F1。
 knn_predict -> kNN 投票。
 stable_softmax -> 稳定 softmax。

常见坑:

- precision 分母是预测为正, recall 分母是真实为正。
- exp(x) 可能溢出, softmax 要先减最大值。
- kNN 平票规则按题面, 没说时可取标签编号小者。
- 归一化时最大值等于最小值要特判。

暴力/部分分替代:

- 不会训练模型时, 先写最近邻、多数类、线性打分 baseline。
- 不会复杂指标时, 先输出混淆矩阵和 accuracy。
- 聚类不会收敛判断时, 按题面固定迭代次数。

1. 监督学习和评估指标

概念	说明
feature	样本输入变量
label	真实类别或目标值
train/test	训练集/测试集
overfit	训练好、测试差
underfit	训练和测试都差
accuracy	$(TP+TN)/(TP+TN+FP+FN)$
precision	$TP/(TP+FP)$
recall	$TP/(TP+FN)$
F1	$2PR/(P+R)$
MSE	平方误差均值
MAE	绝对误差均值

```
struct BinaryMetric {
    int tp = 0, fp = 0, fn = 0, tn = 0;
    double accuracy, precision, recall, f1;
};

BinaryMetric binary_metrics(const vector<int> &truth, const vector<int> &pred) {
    BinaryMetric r;
    int n = (int)truth.size();
    for (int i = 0; i < n; i++) {
        if (truth[i] == 1 && pred[i] == 1) r.tp++;
        else if (truth[i] == 0 && pred[i] == 1) r.fp++;
        else if (truth[i] == 1 && pred[i] == 0) r.fn++;
        else r.tn++;
    }
    r.accuracy = (double)(r.tp + r.tn) / max(1, n);
    r.precision = (r.tp + r.fp == 0 ? 0 : (double)r.tp / (r.tp + r.fp));
    r.recall = (r.tp + r.fn == 0 ? 0 : (double)r.tp / (r.tp + r.fn));
    r.f1 = (r.precision + r.recall == 0 ? 0 : 2 * r.precision * r.recall / (r.precision + r.recall));
    return r;
}
```

2. 常见模型速查

模型	考场实现
kNN	算距离, 排序, 前 k 个投票
朴素贝叶斯	用 log 概率相加, 避免下溢
线性回归	$y=w \cdot x + b$
Logistic	$p=\text{sigmoid}(w \cdot x+b)$
SVM	margin 与 hinge loss
决策树	Gini 或 entropy 选划分

模型	考场实现
k-means	分配最近中心，再重算中心
DNN 前向	矩阵乘 + 激活函数
Q-learning	$Q = Q + \alpha * (r + \gamma * \max Q_{\text{next}} - Q)$

3. softmax 和 kNN 短代码

```
vector<double> stable_softmax(vector<double> z) {
    double mx = *max_element(z.begin(), z.end());
    double sum = 0;
    for (double &x : z) {
        x = exp(x - mx);
        sum += x;
    }
    for (double &x : z) x /= sum;
    return z;
}

int knn_predict(const vector<vector<double>> &x, const vector<int> &label,
               const vector<double> &q, int k) {
    vector<pair<double, int>> v;
    for (int i = 0; i < (int)x.size(); i++) {
        double d2 = 0;
        for (int j = 0; j < (int)q.size(); j++) {
            double t = x[i][j] - q[j];
            d2 += t * t;
        }
        v.push_back({d2, label[i]});
    }
    sort(v.begin(), v.end());
    map<int, int> cnt;
    int best_label = v[0].second, best_count = 0;
    for (int i = 0; i < k && i < (int)v.size(); i++) {
        int c = ++cnt[v[i].second];
        if (c > best_count || (c == best_count && v[i].second < best_label)) {
            best_count = c;
            best_label = v[i].second;
        }
    }
    return best_label;
}
```

SIGN-NOIP-01 NOIP/CSP 初赛式信息学常识

模块编号: SIGN-NOIP-01

模块名称: NOIP/CSP 初赛式信息学基础常识: 概念题、读程序、复杂度和常识判断

标签: 签到题、NOIP 初赛、CSP 初赛、信息学常识、读程序、复杂度、计算机基础、C++17

一句话用途: 遇到选择/填空风格的计算机常识、复杂度判断、读程序输出、基础概念题时, 用本模块快速排除错误选项和防止签到题失分。

题面触发词:

- 下列说法正确的是、时间复杂度、空间复杂度、输出结果。
- 数据结构、栈、队列、树、图、二叉树遍历。
- 操作系统、网络、数据库、信息安全、编码、ASCII、二进制。
- 算法性质、稳定排序、递归、循环、溢出、短路求值。

什么时候用:

- 题目不像上机编程题, 而像 NOIP/CSP 初赛知识点。
- 题目要求判断概念、估算复杂度或阅读一段短代码。

- 你需要在很短时间内确定常识性结论。

不要什么时候用：

- 需要完整实现算法时，回到前面对应算法卷。
- 需要具体网络协议或 OS API 细节时，按题面给出的规则为准。
- 需要现代 AI 细节时，翻 SIGN-AI-02 或第 10 卷。

复杂度：

- 读程序题按循环嵌套估算。
- 概念题 $O(1)$ 查表。
- 树图概念题按节点边数量判断。

依赖的标准容器：无固定依赖；常用 `vector`、`stack`、`queue`、`set` 辅助模拟。

输入如何整理：

读程序题：

1. 标出变量初值。
2. 标出循环次数。
3. 标出每轮改变哪些变量。
4. 小数据直接手动列表模拟。

接口：

复杂度估算 -> 看循环层数、递归式、排序、图边数。

概念判断 -> 查本模块术语表。

读程序 -> 建表模拟变量变化。

常见坑：

- `&&` 和 `||` 有短路求值。
- `i++` 返回旧值，`++i` 返回新值。
- 整数除法会截断，小数要转 `double`。
- 递归既有时间消耗，也有调用栈空间。
- 稳定排序保持相等关键字原相对顺序。

暴力/部分替代：

- 读程序算不清时，把每轮变量写成表格。
- 复杂度不确定时，先算最内层语句总执行次数。
- 树图题不确定时，画 5 个点以内的例子。

1. 初赛常识速查表

主题	关键结论
冯诺依曼结构	运算器、控制器、存储器、输入设备、输出设备
CPU	负责取指、译码、执行；主频不等于绝对性能
RAM	断电丢失，随机访问
ROM	通常用于固件，断电不丢
Cache	比内存快，比寄存器慢，用局部性提升性能
操作系统	管理进程、内存、文件、设备
编译器	把高级语言翻译成机器可执行程序
解释器	边解释边执行
ASCII	7 bit 基本编码，常用字符可用 0..127
Unicode	字符集，UTF-8 是一种编码方式
IP	网络层地址
TCP	面向连接、可靠传输
UDP	无连接、不保证可靠，开销较小
HTTP	应用层协议
数据库主键	唯一标识一条记录
排序稳定性	相等元素相对顺序不变

2. 复杂度判断口令

代码形态	复杂度
单循环 $i=1..n$	$O(n)$
双重独立循环	$O(n^2)$
for ($i=1; i \leq n; i*=2$)	$O(\log n)$
外层 n , 内层 $\log n$	$O(n \log n)$
排序	通常 $O(n \log n)$
BFS/DFS 邻接表	$O(n+m)$
Floyd	$O(n^3)$
枚举所有子集	$O(2^n)$
全排列	$O(n!)$
二分查找	$O(\log n)$
递归 $T(n)=T(n/2)+O(1)$	$O(\log n)$
递归 $T(n)=2T(n/2)+O(n)$	$O(n \log n)$

3. 读程序常见语义

语法	规则
$a = b = c$	从右向左赋值
$a += b$	等价于 $a = a + b$
$i++$	表达式值是旧 i , 然后加一
$++i$	先加一, 表达式值是新 i
$\&\&$	左边假则右边不算
$\ \ $	左边真则右边不算
break	跳出最近一层循环
continue	进入下一轮循环
数组越界	C++ 未定义行为, 上机要避免

4. 数据结构概念题

结构	特征
栈	后进先出, 括号匹配、递归调用
队列	先进先出, BFS
优先队列	每次取最小/最大, 堆实现
集合	去重, 判断存在
映射	key -> value
二叉树	每个节点最多两个孩子
满二叉树	每层都满
完全二叉树	最后一层从左到右填
二叉搜索树	左小右大, 中序有序
图	点和边
树	连通无环图, n 点 $n-1$ 边

5. 排序常识

排序	平均复杂度	稳定性
冒泡排序	$O(n^2)$	稳定
插入排序	$O(n^2)$	稳定
选择排序	$O(n^2)$	通常不稳定
快速排序	$O(n \log n)$ 平均	不稳定
归并排序	$O(n \log n)$	稳定
堆排序	$O(n \log n)$	不稳定
计数排序	$O(n+V)$	可稳定

SIGN-NOIP-02 读程序、流程图、伪代码与基础程序语义

模块编号: SIGN-NOIP-02

模块名称: 读程序和流程图速查: 变量表、循环次数、递归栈、伪代码、运算符优先级和输出判断

标签: 签到题、NOIP 初赛、CSP 初赛、读程序、流程图、伪代码、程序语义、C++17

一句话用途: 当题目要求阅读一段程序、判断流程图输出、把伪代码翻译成代码或分析基础程序语义时, 用本模块按固定步骤模拟, 避免凭感觉丢签到分。

题面触发词:

- 阅读程序, 写出运行结果。
- 流程图、开始/结束、判断框、处理框、输入输出框。
- 伪代码、算法描述、循环变量、递归调用。
- `i++`、`++i`、短路求值、运算符优先级。
- 函数传参、局部变量、全局变量、递归返回。

什么时候用:

- 题目不要求你设计新算法, 只要求模拟已有代码。
- 题目给一段短程序或流程图, 让你填输出。
- 模拟题规则复杂, 先用本模块方法整理状态表。
- NOIP/CSP 初赛式判断题涉及 C++ 基础语义。

不要什么时候用:

- 代码超过几十行且是完整算法时, 应回到对应算法卷理解整体模型。
- 题目使用非 C++ 语言时, 不要套 C++ 特有规则。
- 如果题面明确给了伪代码语义, 以题面规则优先。

复杂度:

- 手动读程序: 按循环总执行次数。
- 流程图模拟: 按路径和循环次数。
- 递归模拟: 按调用树节点数; 小数据画调用栈。

依赖的标准容器:

- 纸上变量表即可。
- 若要写小模拟程序, 常用 `vector`、`stack`、`queue`、`map`。

输入如何整理:

读程序四步:

1. 抄变量初值。
2. 标循环边界和每轮变化。
3. 建表记录每轮关键变量。
4. 只在输出语句处记录输出, 不要脑补。

接口:

`trace_variables(code)` -> 变量表。
`count_loop_times(loop)` -> 循环次数。
`simulate_recursion(f,args)` -> 调用栈/返回值。
`flowchart_to_pseudocode(chart)` -> 按框和箭头翻译。

常见坑:

- `i++` 和 `++i` 在表达式中的值不同。
- `&&` 左侧为假时右侧不执行, `||` 左侧为真时右侧不执行。
- `=` 是赋值, `==` 是比较。
- `/` 对整数是整除, `%` 只适合整数。
- `else` 与最近的未匹配 `if` 配对。
- 函数局部变量每次调用都有自己的一份。
- 数组下标从 0 开始是 C++ 语义, 但本资料算法模板统一偏向 1-index; 读别人代码时按代码本身。

暴力/部分替代:

- 手算困难时, 在草稿纸上列 5 到 10 行变量表。
- 递归困难时画调用树, 并标每个调用的参数和返回值。
- 流程图困难时把每个框编号, 沿箭头一步步走。
- 选择题不确定时先排除违反短路、整除、循环次数的选项。

1. 流程图符号

符号	含义
圆角矩形/椭圆	开始或结束
平行四边形	输入或输出
矩形	处理、赋值、计算
菱形	判断条件, 分 Yes/No
箭头	控制流方向

口令:
流程图就是没有语法糖的程序。
菱形对应 if/while 条件。
回到前面的一条箭头通常表示循环。

2. 变量表模板

建议画表:

步骤	条件	i	j	ans	输出
初始	-				
1					
2					

不要把所有变量都抄进去，只保留会影响分支、循环和输出的变量。

3. 循环次数速查

循环	执行次数
for (int i=1;i<=n;i++)	n
for (int i=0;i<n;i++)	n
for (int i=1;i<n;i++)	n-1
for (int i=n;i>=1;i--)	n
for (int i=1;i<=n;i*=2)	floor(log2 n)+1
while (x>0) x/=10	十进制位数
while (x) x-=x&-x	x 的二进制 1 的个数

双层循环要看内层是否依赖外层:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        cnt++;
    }
}
```

总次数:
 $1 + 2 + \dots + n = n(n+1)/2$

4. C++ 运算符优先级高频版

从高到低记常用部分:

优先级	运算符
高	!, ++, --, 一元负号
	*, /, %
	+, -
	<, <=, >, >=
	==, !=
	&&

优先级	运算符
低	=、+=、-=

考场建议：

读题时按优先级算。

写代码时主动加括号，降低心智负担。

5. 自增自减

```
int i = 3;
int a = i++; // a=3, i=4
int b = ++i; // i=5, b=5
```

不要在同一表达式里多次修改同一个变量，例如：

```
int x = i++ + ++i; // 不建议，读程序题也要谨慎按语法规则
```

竞赛写代码时避免这种写法。读程序题若出现，优先看题面是否规定语言版本和行为；若是未定义行为，通常不会作为严肃考点。

6. 短路求值

```
if (p != 0 && x / p > 3) {
    // p==0 时右边不会算，避免除以 0
}
```

规则：

表达式	左边情况	右边是否执行
A && B	A 为假	不执行
A && B	A 为真	执行
A B	A 为真	不执行
A B	A 为假	执行

读程序时，右侧如果有 i++、函数调用、输出，必须考虑是否真的执行。

7. 函数传参

C++ 常见：

```
void f1(int x) { x = 10; } // 值传递，外面不变
void f2(int &x) { x = 10; } // 引用传递，外面改变
void f3(int a[]) { a[1] = 10; } // 数组传入后可改原数组
```

最小示例：

```
#include <bits/stdc++.h>
using namespace std;

void add_value(int x) {
    x++;
}

void add_ref(int &x) {
    x++;
}

int main() {
    int a = 5;
    add_value(a);
    cout << a << "\n"; // 5
    add_ref(a);
    cout << a << "\n"; // 6
}
```

```

    return 0;
}

```

8. 递归调用栈

读递归三问：

1. 递归终止条件是什么？
2. 每次调用参数怎么变？
3. 返回后还做什么？

例子：

```

int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 2);
}

```

f(4) 调用树：

```

f(4)
  f(3)
    f(2)
      f(1)=1
      f(0)=1
    f(1)=1
  f(2)
    f(1)=1
    f(0)=1

```

所以 f(4)=5。

9. 递归输出顺序

```

void g(int n) {
    if (n == 0) return;
    cout << n << " ";
    g(n - 1);
    cout << n << " ";
}

```

g(3) 输出：

3 2 1 1 2 3

口令：

递归调用前的输出：从大到小。

递归返回后的输出：从小到大。

10. 伪代码翻译规则

伪代码	C++
x <- y	x = y;
for i = 1 to n	for (int i=1;i<=n;i++)
while condition	while (condition)
if condition then	if (condition)
return x	return x;
and/or/not	&& / / !

注意：

伪代码数组可能从 1 开始，C++ vector 默认 0 开始。

为了和本资料模板统一，自己实现时优先开 n+1 用 1-index。

11. 分支配对

```
if (a)
    if (b) x = 1;
    else x = 2;
```

else 与最近的未匹配 if (b) 配对, 不是 if (a)。

建议写法:

```
if (a) {
    if (b) x = 1;
    else x = 2;
}
```

12. 二维数组和循环顺序

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        cin >> a[i][j];
    }
}
```

含义:

i 通常是行, j 通常是列。
先读第 1 行从左到右, 再读第 2 行。

坐标题常用方向数组:

```
int dx[5] = {0, -1, 1, 0, 0};
int dy[5] = {0, 0, 0, -1, 1};
```

13. 读程序完整示例

题目:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5, ans = 0;
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0) ans += i;
        else ans += i * i;
    }
    cout << ans << "\n";
    return 0;
}
```

变量表:

i	奇偶	加多少	ans
1	奇	1	1
2	偶	2	3
3	奇	9	12
4	偶	4	16
5	奇	25	41

输出:

41

14. 看输出格式

代码	输出效果
<code>cout << x;</code>	不自动换行
<code>cout << x << "\n";</code>	输出后换行
<code>cout << a << " " << b;</code>	中间一个空格
<code>cout << fixed << setprecision(2) << x;</code>	保留 2 位小数

读程序题要保留空格和换行。选择题若只问数值，通常忽略末尾空格；上机提交时不要多输出调试信息。

15. 代码阅读排错清单

- 循环从 0 还是 1 开始？
- 结束条件是 $< n$ 还是 $\leq n$ ？
- 变量是否在循环内部重新初始化？
- `break/continue` 跳到哪里？
- 右侧表达式是否因为短路没有执行？
- 函数改的是副本还是原变量？
- 递归返回后还有没有语句？
- 输出语句执行了几次？

SIGN-OSNET-01 操作系统、网络与数据库常识

模块编号: SIGN-OSNET-01

模块名称: 操作系统、计算机网络、数据库和 Web 常识

标签: 签到题、操作系统、网络、数据库、Web、SQL、HTTP、进程线程、C++17

一句话用途: 信息学/AI 招生题可能把 OS、网络、数据库和 Web 概念作为背景，本模块用于快速识别术语和常见计算。

题面触发词: 进程、线程、死锁、内存、文件系统、TCP、UDP、HTTP、IP、DNS、SQL、数据库、事务。

什么时候用:

- 题目是计算机常识判断或简单模拟。
- 题目给网络带宽、延迟、请求数、数据库表格，要求计算或筛选。
- 题目涉及 SQL 的选择、过滤、排序、分组概念。

不要什么时候用:

- 需要真实系统调用或网络编程，考试一般不会要求。
- 复杂 SQL 优化和数据库事务细节，本卷只作常识。
- IP 位运算具体代码翻 SIGN-CS-01。

复杂度:

- 概念判断 $O(1)$ 。
- 表格筛选排序按 $O(n)$ 或 $O(n \log n)$ 。
- 网络传输按字节数和带宽估算。

依赖的标准容器: `vector`、`map`、`set`、`sort`、`string`。

输入如何整理:

网络题先统一 `bps/B/s`。

数据库题先明确表头、筛选条件、排序键、聚合字段。

OS 题先分清进程、线程、程序。

接口:

OS 概念 -> 查表。

网络计算 -> $\text{time} = \text{data} / \text{bandwidth} + \text{latency}$ 。

SQL 模拟 -> `filter` -> `group` -> `sort` -> `output`。

常见坑:

- 程序是静态文件，进程是运行中的程序实例。
- 线程共享同一进程地址空间，进程之间相对隔离。
- TCP 可靠有连接，UDP 简单低开销但不保证可靠。

- DNS 负责域名到 IP 的解析。
- SQL 的 WHERE 在分组前过滤，HAVING 在分组后过滤。

暴力/部分替代：

- SQL 题不会写抽象查询时，按行模拟筛选。
- 网络题协议复杂时，按题面公式和单位计算。
- 资源调度题数据小可逐时间片模拟。

1. 操作系统常识

概念	说明
程序	存在磁盘上的代码和数据
进程	程序的一次运行实例
线程	进程内的执行流
并发	多任务交替推进
并行	多任务同时执行
死锁	多个任务互相等待资源
虚拟内存	给进程提供连续地址空间的抽象
页	内存管理的固定大小块
文件系统	管理文件命名、目录、权限、存储
调度	决定哪个任务运行

死锁四个必要条件：

互斥、占有且等待、不可抢占、循环等待。

2. 网络常识

层/协议	作用
IP	寻址和路由
TCP	可靠字节流，面向连接
UDP	无连接报文
DNS	域名解析
HTTP	Web 请求响应
HTTPS	HTTP + TLS 加密
URL	资源定位符
CDN	内容分发网络

HTTP 状态码：

范围	含义
2xx	成功
3xx	重定向
4xx	客户端错误
5xx	服务器错误

3. 数据库和 SQL

概念	说明
表	行和列组成
行/记录	一条数据
列/字段	一个属性
主键	唯一标识记录
外键	指向另一表主键
索引	加速查找的数据结构
事务	一组操作作为整体
ACID	原子性、一致性、隔离性、持久性

SQL 执行直觉：

FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT

模拟 SQL:

```
struct Row {
    string name;
    int score;
};

vector<Row> filter_sort(vector<Row> rows, int low) {
    vector<Row> v;
    for (auto r : rows) {
        if (r.score >= low) v.push_back(r);
    }
    sort(v.begin(), v.end(), [](const Row &a, const Row &b) {
        if (a.score != b.score) return a.score > b.score;
        return a.name < b.name;
    });
    return v;
}
```

SIGN-PROB-01 概率统计速查

模块编号: SIGN-PROB-01

模块名称: 概率、期望、统计指标和描述统计

标签: 签到题、概率、统计、期望、方差、分位数、Bayes、C++17

一句话用途: 遇到概率、统计、平均数、方差、分位数、相关系数或 A/B 测试类题时, 用本模块快速查公式和小代码。

题面触发词: 概率、条件概率、Bayes、期望、方差、标准差、中位数、众数、分位数、相关系数、直方图。

什么时候用:

- 题目给一组数, 要求统计量。
- 题目给事件概率, 要求组合概率或条件概率。
- 机器学习评估题需要先算基础统计。

不要什么时候用:

- 概率 DP 状态复杂, 优先 DP 卷。
- 组合概率涉及大组合数取模, 优先第 8 卷。
- 随机模拟只能做调试或部分分, 不应替代精确算法。

复杂度:

- 均值/方差: $O(n)$ 。
- 中位数/分位数: 排序 $O(n \log n)$, 或 `nth_element` 平均 $O(n)$ 。
- 相关系数: $O(n)$ 。

依赖的标准容器: `vector<double>`、`vector<int>`、`sort`、`map`。

输入如何整理:

先确认统计的是总体还是样本。

总体方差分母 n , 样本方差分母 $n-1$ 。

百分位定义题面可能不同, 按题面为准。

接口:

`mean(a,n)` -> 平均数。

`variance_population(a,n)` -> 总体方差。

`median(a,n)` -> 中位数。

`pearson(x,y,n)` -> 皮尔逊相关系数。

常见坑:

- 方差不要忘记平方。
- $n=1$ 时样本方差分母 $n-1$ 为 0, 要特判。

- 概率相乘需要独立性；不独立时用条件概率。
- 精度输出用 double，计数用 long long。

暴力/部分替代：

- 概率推不出时，小状态可枚举所有结果。
- 分位数规则不确定时，优先按题面样例反推。
- 大样本统计不会优化时，先排序写 $O(n \log n)$ 。

1. 概率公式

名称	公式
补事件	$P(\text{not } A) = 1 - P(A)$
加法公式	$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$
条件概率	$P(A B) = P(A \text{ and } B) / P(B)$
乘法公式	$P(A \text{ and } B) = P(A B) * P(B)$
独立事件	$P(A \text{ and } B) = P(A) * P(B)$
Bayes	$P(A B) = P(B A)P(A) / P(B)$
期望线性性	$E(X+Y) = E(X) + E(Y)$
方差	$E(X^2) - E(X)^2$

2. 描述统计小代码

```
double mean_value(const vector<double> &a) {
    double s = 0;
    for (double x : a) s += x;
    return s / (double)a.size();
}

double variance_population(const vector<double> &a) {
    double mu = mean_value(a), s = 0;
    for (double x : a) s += (x - mu) * (x - mu);
    return s / (double)a.size();
}

double median_value(vector<double> a) {
    sort(a.begin(), a.end());
    int n = (int)a.size();
    if (n % 2 == 1) return a[n / 2];
    return (a[n / 2 - 1] + a[n / 2]) / 2.0;
}
```

3. 相关系数

```
double pearson(const vector<double> &x, const vector<double> &y) {
    int n = (int)x.size();
    double mx = mean_value(x), my = mean_value(y);
    double num = 0, sx = 0, sy = 0;
    for (int i = 0; i < n; i++) {
        double dx = x[i] - mx, dy = y[i] - my;
        num += dx * dy;
        sx += dx * dx;
        sy += dy * dy;
    }
    if (sx == 0 || sy == 0) return 0;
    return num / sqrt(sx * sy);
}
```

4. 常见分布

分布	使用场景	关键量
Bernoulli	一次成败	$P(1)=p$
Binomial	n 次独立成败	$C(n,k)p^k(1-p)^{(n-k)}$
Geometric	第一次成功在第几次	$(1-p)^{(k-1)}p$
Poisson	单位时间稀有事件数	$\lambda^k e^{-\lambda} / k!$
Normal	近似连续测量误差	z-score

SIGN-SEC-01 信息安全、校验、压缩与信息论常识

模块编号: SIGN-SEC-01

模块名称: 信息安全、哈希、校验、压缩和信息论基础

标签: 签到题、信息安全、哈希、校验和、压缩、熵、编码、CRC、C++17

一句话用途: 遇到校验码、哈希、加密、压缩率、熵、错误检测等常识题时, 用本模块快速查概念和小公式。

题面触发词: 校验和、奇偶校验、CRC、哈希、MD5、SHA、加密、压缩率、熵、Huffman、错误检测。

什么时候用:

- 题目考概念判断: 哈希 vs 加密, 压缩 vs 编码。
- 题目要求计算简单校验和、奇偶校验、压缩率。
- 题目给频率, 要求估算信息熵或 Huffman 直觉。

不要什么时候用:

- 不要自己实现真实密码算法, 考试一般不会要求。
- 如果题目给了具体编码树, 按题面模拟, 不要套概念。
- 大规模字符串哈希算法题翻字符串卷。

复杂度:

- 简单校验: $O(n)$ 。
- 频率统计: $O(n)$ 。
- Huffman 建树: $O(k \log k)$ 。

依赖的标准容器: string、vector<int>、priority_queue、map。

输入如何整理:

先确认处理单位是 bit、byte 还是字符。

压缩率常见写法:

compressed / original 或 (original-compressed)/original, 按题面。

接口:

xor_checksum(s) -> 异或校验。

parity(x) -> 二进制 1 的个数奇偶。

entropy(freq) -> 信息熵。

常见坑:

- 哈希不是加密; 哈希通常不可逆, 加密应可用密钥解密。
- 编码不是压缩; Base64 反而会变大。
- 压缩后大小可能因为头部信息而变大。
- 奇偶校验只能检测奇数个位错误, 不一定能纠错。

暴力/部分替代:

- 复杂 CRC 不会时, 先按题面给的小规则逐位模拟。
- Huffman 不会时, 小数据可以枚举树形很难, 优先掌握优先队列贪心。

1. 概念区分

概念	目的	是否可逆
编码	表示数据	通常可逆
压缩	减少大小	无损可逆, 有损不可完全恢复

概念	目的	是否可逆
哈希	摘要/查找/完整性	通常不可逆
加密	保密	有密钥可逆
签名	证明身份和完整性	验证可行，不是解密
校验	检测错误	通常不能恢复

2. 校验短代码

```
int parity_ones(unsigned int x) {
    return __builtin_popcount(x) & 1;
}

unsigned char xor_checksum(const string &s) {
    unsigned char ans = 0;
    for (unsigned char c : s) ans ^= c;
    return ans;
}

int digit_sum_mod10(const string &s) {
    int sum = 0;
    for (char c : s) if (isdigit((unsigned char)c)) sum = (sum + c - '0') % 10;
    return sum;
}
```

3. 压缩和熵

信息熵:

$$H = -\sum p_i * \log_2(p_i)$$

直觉:

- 越均匀，熵越大。
- 越集中，越容易压缩。
- Huffman 编码给高频字符更短码。

```
double entropy_from_counts(const vector<int> &cnt) {
    double total = 0;
    for (int x : cnt) total += x;
    double h = 0;
    for (int x : cnt) {
        if (x == 0) continue;
        double p = x / total;
        h -= p * log2(p);
    }
    return h;
}
```

4. 常见压缩率

问法	公式
压缩后占原来的比例	compressed/original
压缩率减少了多少	(original-compressed)/original
压缩倍数	original/compressed
Base64 大小	约 ceil(n/3)*4 byte

SIGN-SIM-01 生活化签到模拟题模板

模块编号: SIGN-SIM-01

模块名称: 生活化模拟题: 日期、BMI、排名、Excel 列号、格式和小工具

标签：签到题、生活模拟、日期、BMI、排名、Excel 列号、格式化、C++17

一句话用途：把最常见的生活化签到题做成可抄模板，重点保证输入输出合法、边界不丢分。

题面触发词：BMI、成绩等级、GPA、排名、同分、Excel 列号、日期差、星期、单位换算、括号匹配、状态机。

什么时候用：

- 题目是现实规则模拟，算法不难但规则细。
- 输出格式要求固定小数、补零、对齐或分类文字。
- 需要把字符串编号转换成数字，或把数字转编号。

不要什么时候用：

- 日期题涉及历史儒略历/格里高利历切换、夏令时数据库，优先 SIM-06。
- 表达式、JSON、脚本等复杂解析，优先 SIM-03/04/05。
- 方程求解优先 SIM-07。

复杂度：多数 $O(1)$ ；字符串扫描 $O(\text{len})$ ；排名排序 $O(n \log n)$ 。

依赖的标准容器：string、vector、algorithm、stack、iomanip。

输入如何整理：

先读规则，再把每条规则写成 if/else 或小函数。

有多组数据时每组清空状态。

涉及格式输出时统一放到最后输出。

接口：

bmi(weight,height) -> BMI。
excel_col_to_num(s) -> A1 风格列号转数字。
excel_num_to_col(x) -> 数字转列号。
days_from_civil(y,m,d) -> 日期转序号。
rank_with_ties(score) -> 同分排名。

常见坑：

- 身高若输入厘米，BMI 要除以 100 转米。
- Excel 列号是 1-index: A=1, Z=26, AA=27。
- 排名有 dense ranking 和 competition ranking，按题面。
- 日期差是否包含起止当天，要看题面样例。

暴力/部分替代：

- 日期公式忘记时，小范围逐日加。
- 排名规则复杂时，先排序输出普通名次。
- 状态机不会抽象时，用 if/else 按字符扫描。

1. 高频短规则

模型	规则
BMI	$\text{weight_kg} / \text{height_m}^2$
成绩等级	从高到低写 if，避免区间重叠
GPA	加权平均: $\text{sum}(\text{score} * \text{credit}) / \text{sum}(\text{credit})$
同分排名	比自己分高的人数 + 1
Excel 列号	26 进制但没有 0
括号匹配	栈
自动机	$\text{state} = \text{trans}[\text{state}][\text{input}]$

2. 完整可运行小工具

这个程序故意覆盖多个签到常识：BMP 大小、三角形面积、日期差、二分类指标、进制转换、BMI、Excel 列号。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const double PI = acos(-1.0);
```

```

bool triangle_ok(double a, double b, double c) {
    return a + b > c && a + c > b && b + c > a;
}

double triangle_area(double a, double b, double c) {
    if (!triangle_ok(a, b, c)) return -1.0;
    double s = (a + b + c) / 2.0;
    return sqrt(max(0.0, s * (s - a) * (s - b) * (s - c)));
}

ll bmp_pixel_bytes(ll w, ll h, int bpp) {
    ll row_bits = w * bpp;
    ll row_bytes = ((row_bits + 31) / 32) * 4;
    return row_bytes * h;
}

ll days_from_civil(int y, int m, int d) {
    y -= m <= 2;
    const int era = (y >= 0 ? y : y - 399) / 400;
    const unsigned yoe = (unsigned)(y - era * 400);
    const unsigned doy = (153 * (m + (m > 2 ? -3 : 9)) + 2) / 5 + d - 1;
    const unsigned doe = yoe * 365 + yoe / 4 - yoe / 100 + doy;
    return era * 146097LL + (ll)doe - 719468LL;
}

int hex_value(char c) {
    if ('0' <= c && c <= '9') return c - '0';
    if ('a' <= c && c <= 'f') return c - 'a' + 10;
    if ('A' <= c && c <= 'F') return c - 'A' + 10;
    return -1;
}

ll to_decimal(const string &s, int base) {
    ll ans = 0;
    for (char c : s) ans = ans * base + hex_value(c);
    return ans;
}

ll excel_col_to_num(const string &s) {
    ll ans = 0;
    for (char c : s) ans = ans * 26 + (c - 'A' + 1);
    return ans;
}

string excel_num_to_col(ll x) {
    string s;
    while (x > 0) {
        x--;
        s.push_back(char('A' + x % 26));
        x /= 26;
    }
    reverse(s.begin(), s.end());
    return s;
}

void solve_metrics() {
    int n;
    cin >> n;
    int tp = 0, fp = 0, fn = 0, tn = 0;
    for (int i = 1; i <= n; i++) {
        int y, p;
        cin >> y >> p;
        if (y == 1 && p == 1) tp++;
    }
}

```

```

        else if (y == 0 && p == 1) fp++;
        else if (y == 1 && p == 0) fn++;
        else tn++;
    }
    double acc = (double)(tp + tn) / max(1, n);
    double precision = (tp + fp == 0 ? 0 : (double)tp / (tp + fp));
    double recall = (tp + fn == 0 ? 0 : (double)tp / (tp + fn));
    double f1 = (precision + recall == 0 ? 0 : 2 * precision * recall / (precision + recall));
    cout << fixed << setprecision(6) << acc << ' ' << precision << ' ' << recall << ' ' << f1 << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string mode;
    cin >> mode;
    cout << fixed << setprecision(6);

    if (mode == "triangle") {
        double a, b, c;
        cin >> a >> b >> c;
        double area = triangle_area(a, b, c);
        if (area < 0) cout << "INVALID\n";
        else cout << area << "\n";
    } else if (mode == "bmp") {
        ll w, h;
        int bpp;
        cin >> w >> h >> bpp;
        cout << bmp_pixel_bytes(w, h, bpp) << "\n";
    } else if (mode == "datediff") {
        int y1, m1, d1, y2, m2, d2;
        cin >> y1 >> m1 >> d1 >> y2 >> m2 >> d2;
        cout << llabs(days_from_civil(y1, m1, d1) - days_from_civil(y2, m2, d2)) << "\n";
    } else if (mode == "metrics") {
        solve_metrics();
    } else if (mode == "base") {
        string s;
        int b;
        cin >> s >> b;
        cout.unsetf(ios::floatfield);
        cout << to_decimal(s, b) << "\n";
    } else if (mode == "bmi") {
        double kg, cm;
        cin >> kg >> cm;
        double h = cm / 100.0;
        cout << kg / (h * h) << "\n";
    } else if (mode == "excel_to_num") {
        string s;
        cin >> s;
        cout.unsetf(ios::floatfield);
        cout << excel_col_to_num(s) << "\n";
    } else if (mode == "excel_to_col") {
        ll x;
        cin >> x;
        cout << excel_num_to_col(x) << "\n";
    }
    return 0;
}

```

3. 最小测试样例

triangle

3 4 5

=> 6.000000

bmp

3 2 24

=> 24

datediff

2024 2 28 2024 3 1

=> 2

excel_to_num

AA

=> 27